MARKETING CAMPAIGN PERFORMANCE INSIGHTS

Python Data Analysis and Exploratory Data Visualization

Dataset: marketing_campaign.csv

Tools: Python, Pandas, Matplotlib and Seaborn

Assignment Title: Marketing Campaign Performance Insights

Name: ANCY.V

Course Name: AI DRIVEN DATA ANALYTICS

Submission Date: 11/11/2026

Github Link:

1. Introduction

This project analyses a marketing campaign dataset to understand campaign performance, customer segments, channel effectiveness, time-based trends and geographic patterns. Python libraries are used to load, inspect, summarize and visualize the dataset.

The analysis is organized into three stages: loading the dataset, performing descriptive analysis, and creating exploratory data analysis (EDA) visualizations.

2. Dataset and Tools

Dataset source: https://raw.githubusercontent.com/ArchanaInsights/Datasets/main/marketing_campaign.csv

Tools and libraries: Python, Pandas, Matplotlib and Seaborn.

The dataset contains 22,029 rows and 16 columns. The actual dataset column name is **Channel_Used** (singular), and this name is used in the Python code below.

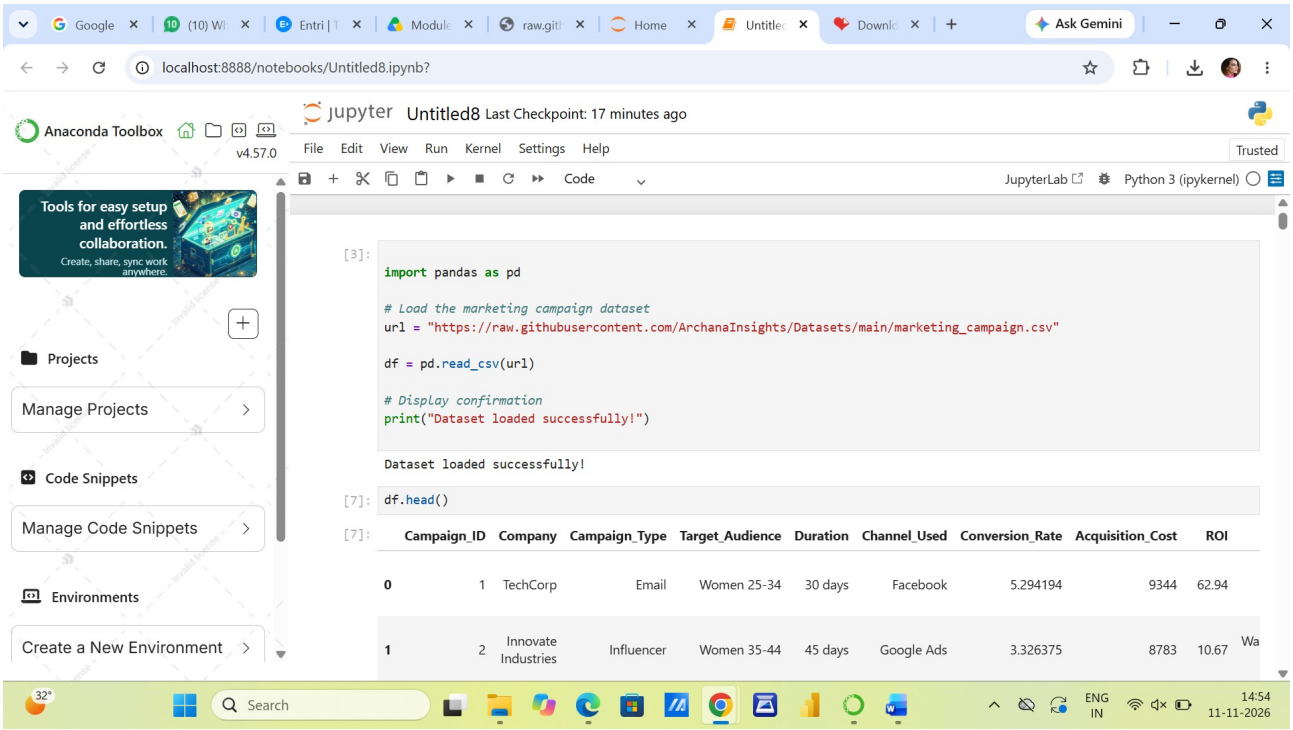
3. Load the Dataset

The CSV file is read from the provided URL into a Pandas DataFrame named **df**.

```
import pandas as pd

url = "https://raw.githubusercontent.com/ArchanaInsights/Datasets/main/marketing_campaign.csv"
df = pd.read_csv(url)

print("Dataset loaded successfully!")
```

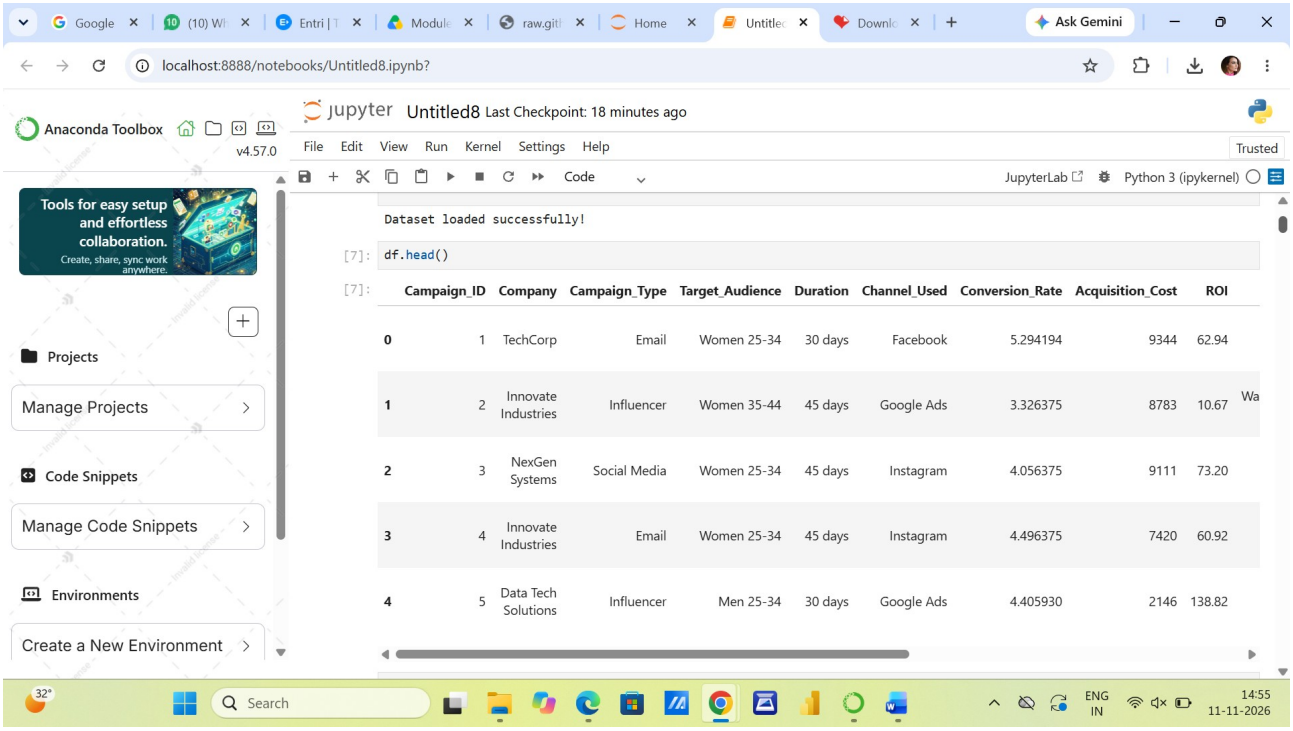


4. Descriptive Analysis

4.1 Display the first few rows

```
df.head()
```

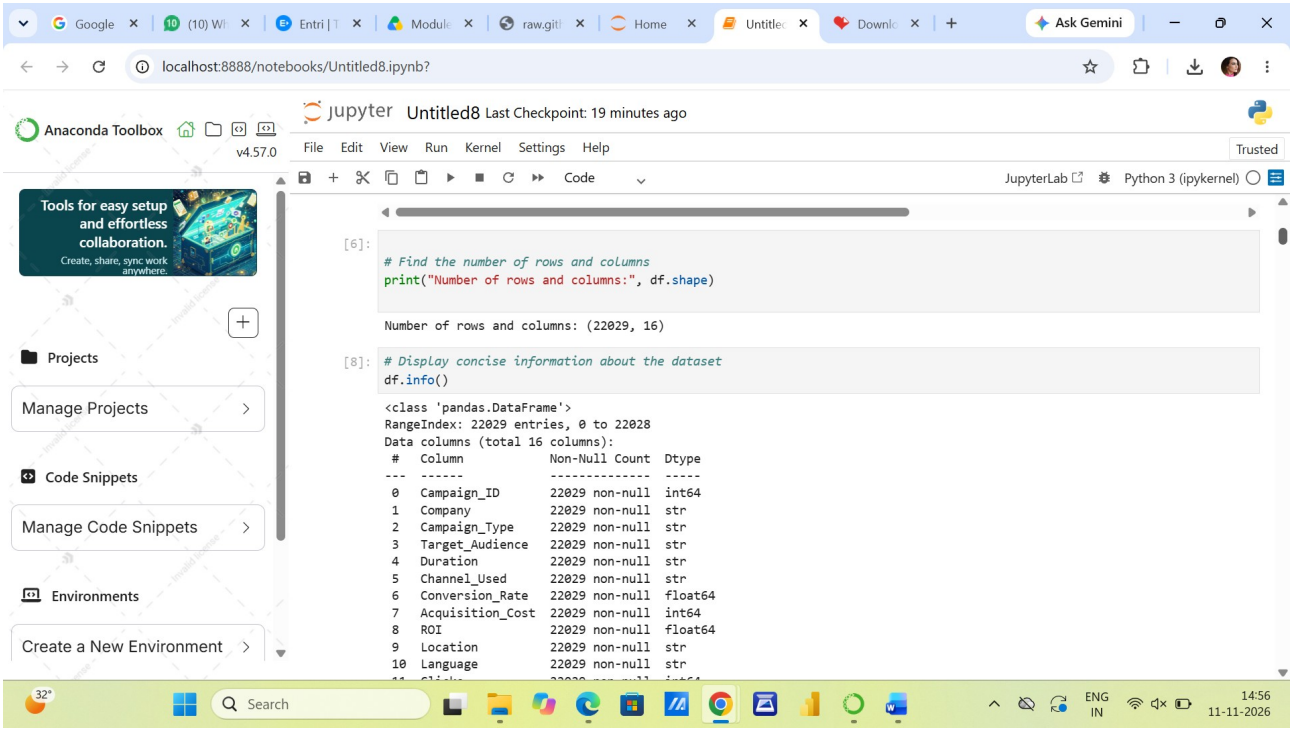
This displays the first five records of the dataset to inspect the structure and sample values.



4.2 Find the number of rows and columns

```
print("Number of rows and columns:", df.shape)
```

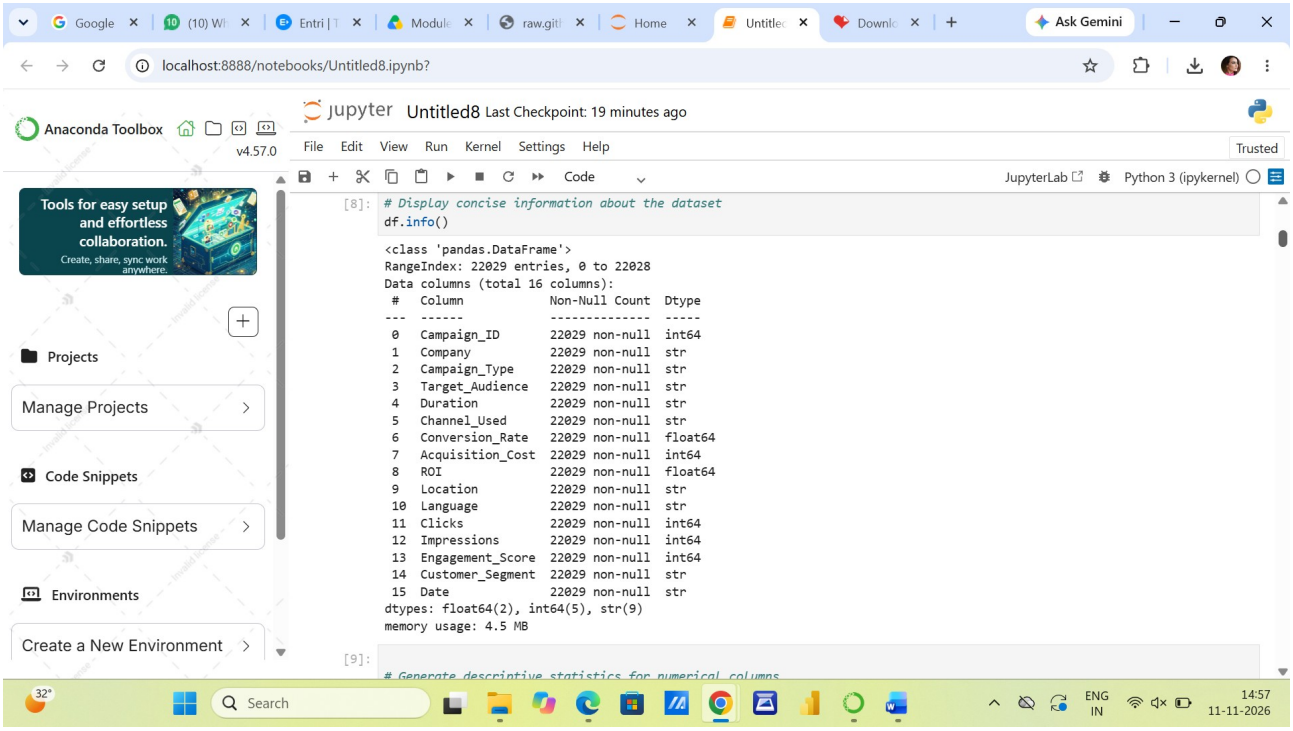
Output: (22029, 16) — 22,029 rows and 16 columns.



4.3 Display the concise dataset summary

```
df.info()
```

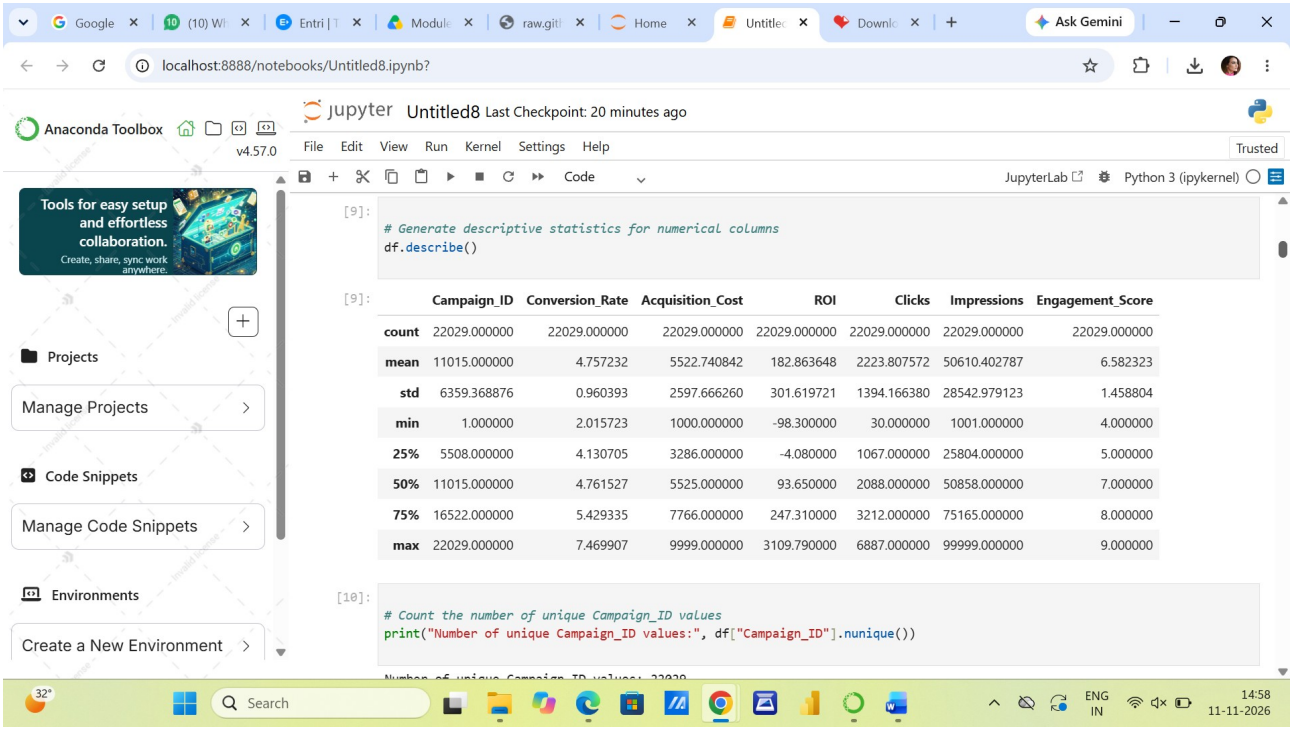
The dataset contains 22,029 non-null values in each of the 16 columns. The columns include 5 integer columns, 2 floating-point columns and 9 text/string columns. The Date column was initially read as text.



4.4 Generate descriptive statistics

```
df.describe()
```

The descriptive statistics summarize numerical columns using count, mean, standard deviation, minimum, quartiles and maximum.

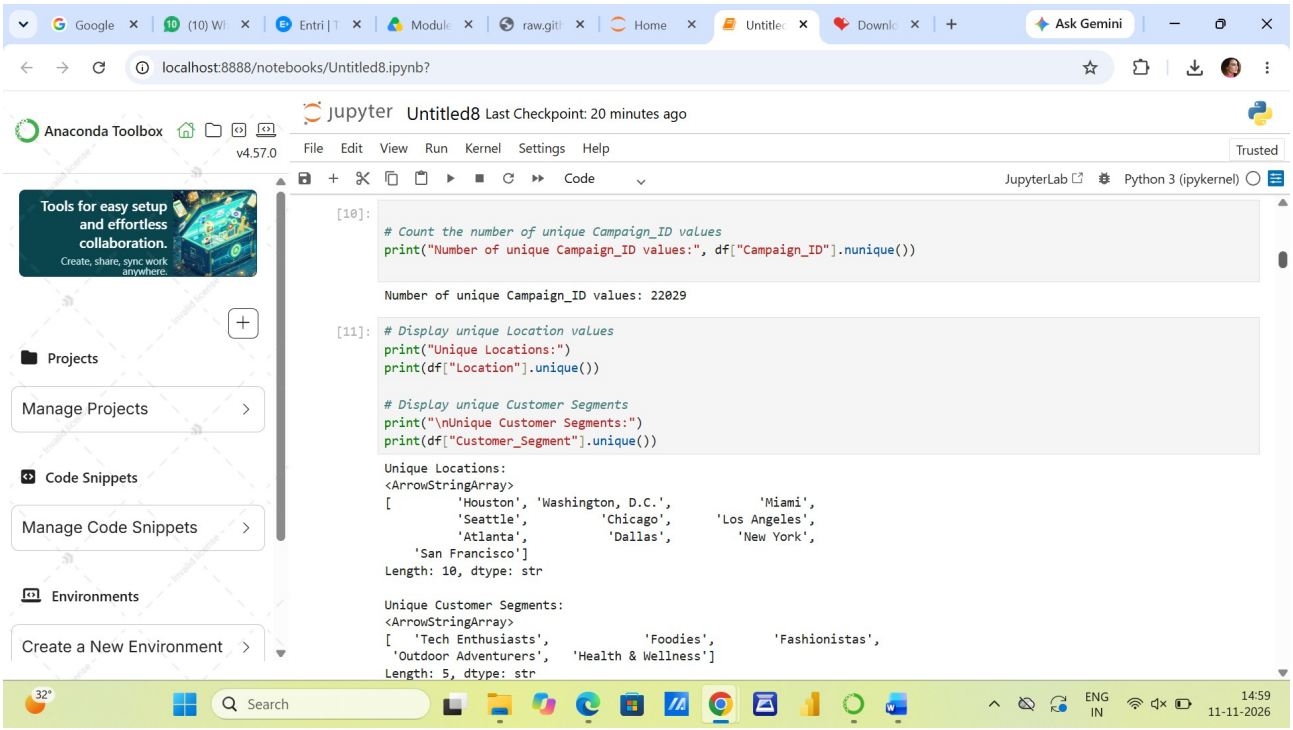


Metric	Conversion_Rate	Acquisition_Cost	ROI
Count	22029	22029	22029
Mean	4.757232	5522.740842	182.863648
Std. deviation	0.960393	2597.666260	301.619721
Minimum	2.015723	1000	-98.30
Median (50%)	4.761527	5525	93.65
Maximum	7.469907	9999	3109.79

4.5 Count unique Campaign_ID values

```
df["Campaign_ID"].nunique()
```

Output: 22029 unique Campaign_ID values.

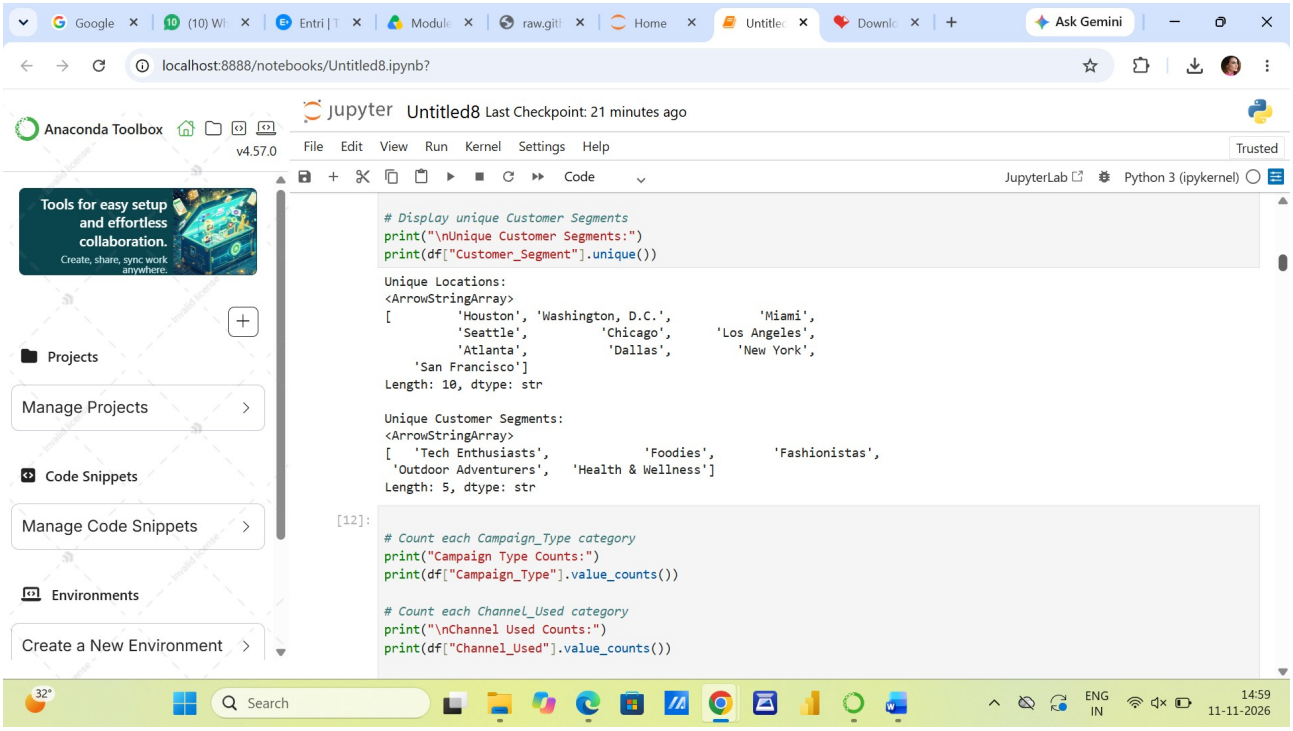


4.6 Display unique Location and Customer_Segment values

```
df["Location"].unique()
df["Customer_Segment"].unique()
```

Locations: Houston, Washington, D.C., Miami, Seattle, Chicago, Los Angeles, Atlanta, Dallas, New York and San Francisco.

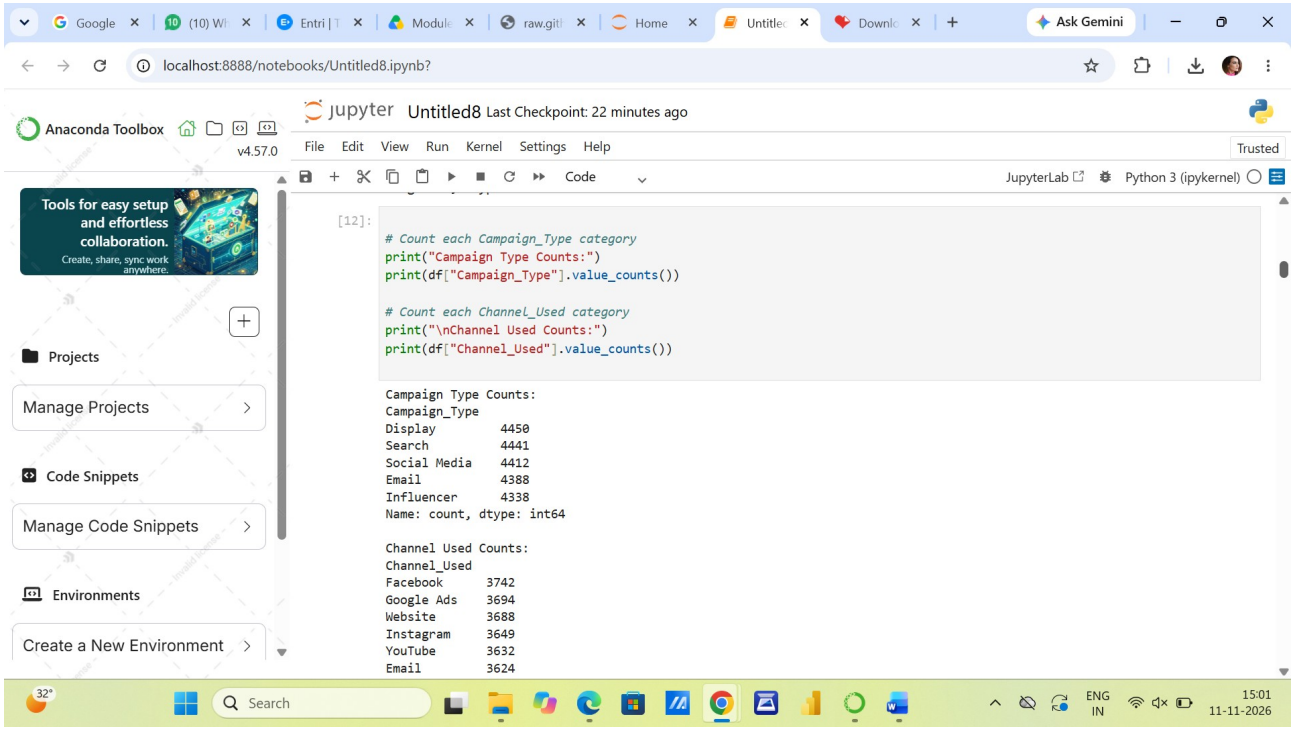
Customer segments: Tech Enthusiasts, Foodies, Fashionistas, Outdoor Adventurers and Health & Wellness.



4.7 Count Campaign_Type and Channel_Used

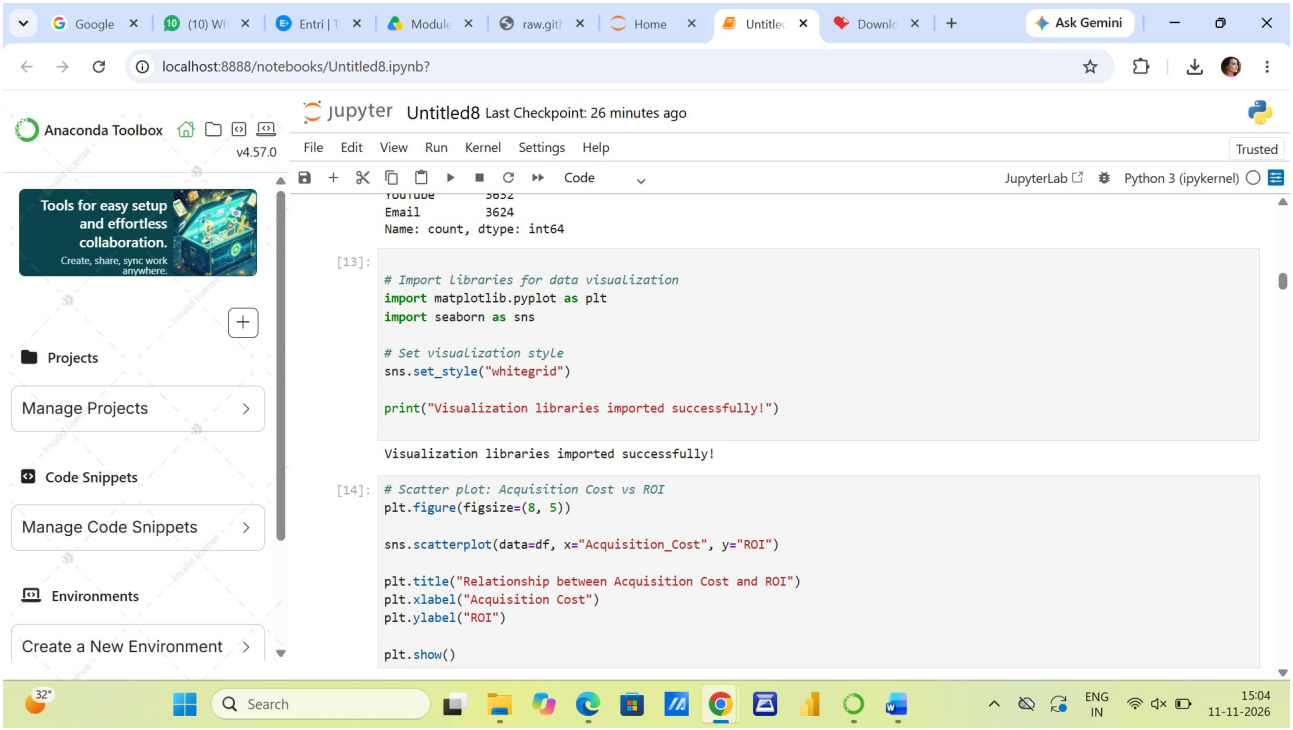
```
df["Campaign_Type"].value_counts()
df["Channel_Used"].value_counts()
```

Campaign_Type	Count	Channel_Used	Count
Display	4450	Facebook	3742
Search	4441	Google Ads	3694
Social Media	4412	Website	3688
Email	4388	Instagram	3649
Influencer	4338	YouTube	3632
		Email	3624


Anaconda Toolbox
v4.57.0
Tools for easy setup and effortless collaboration.
Create, share, sync work anywhere.
Projects
Manage Projects
Code Snippets
Manage Code Snippets
Environments
Create a New Environment
jupyter Untitled8 Last Checkpoint: 22 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[12]:
Count each Campaign_Type category
print("Campaign Type Counts:")
print(df["Campaign_Type"].value_counts())
Count each Channel_Used category
print("\nChannel Used Counts:")
print(df["Channel_Used"].value_counts())
Campaign Type Counts:
Campaign_Type
Display 4450
Search 4441
Social Media 4412
Email 4388
Influencer 4338
Name: count, dtype: int64
Channel Used Counts:
Channel_Used
Facebook 3742
Google Ads 3694
Website 3688
Instagram 3649
YouTube 3632
Email 3624
Search
ENG IN
15:01
11-11-2026

5. Exploratory Data Analysis and Visualization

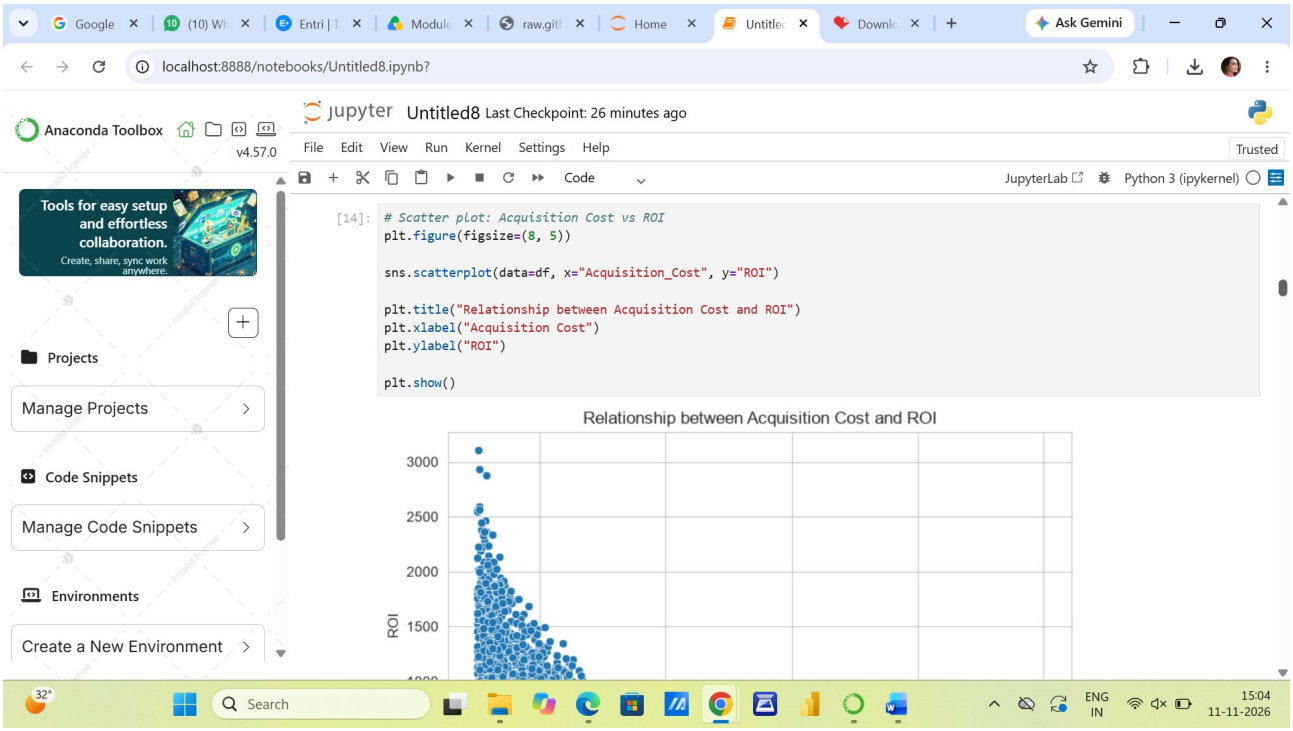
The following visualizations explore relationships and patterns in campaign performance. Each code cell creates one graph.

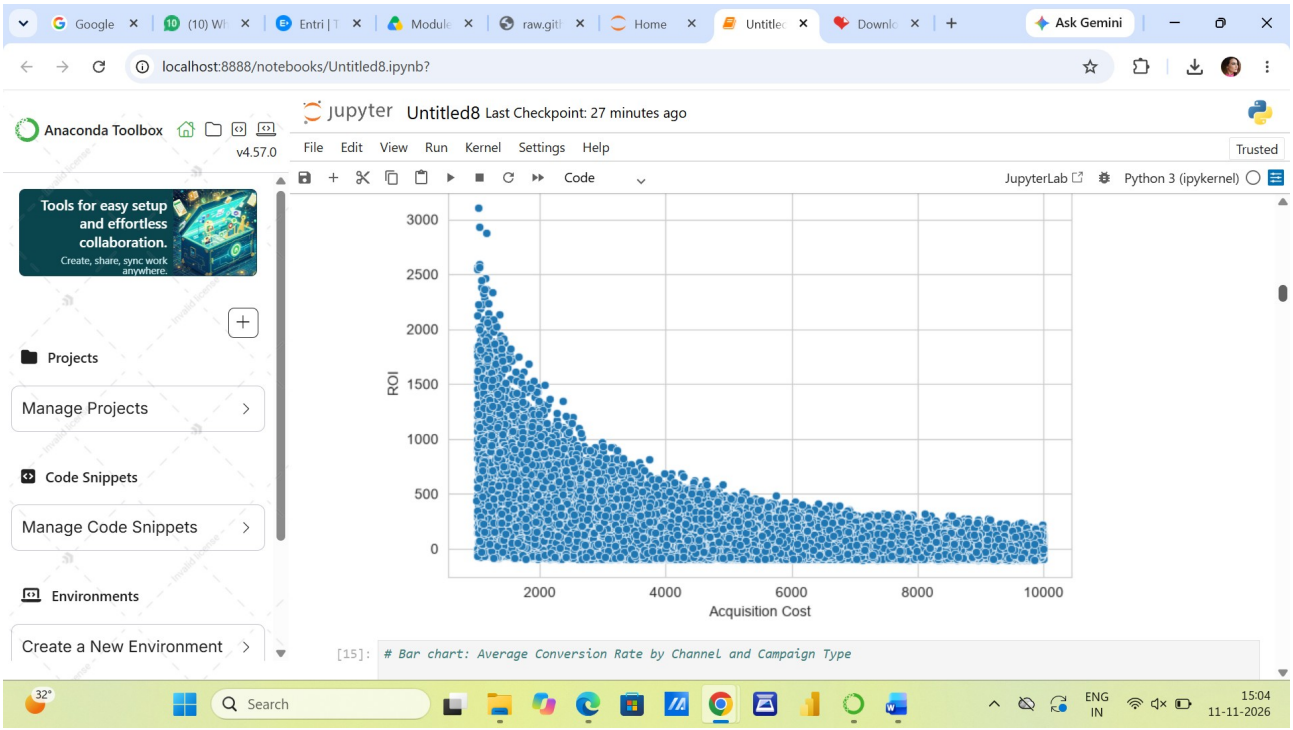


5.1 Scatter Plot – Acquisition Cost vs ROI

Each point represents a campaign. The x-axis shows acquisition cost and the y-axis shows ROI. The plot helps inspect the relationship between campaign cost and return.

```
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x="Acquisition_Cost", y="ROI")
plt.title("Relationship between Acquisition Cost and ROI")
plt.xlabel("Acquisition Cost")
plt.ylabel("ROI")
plt.show()
```

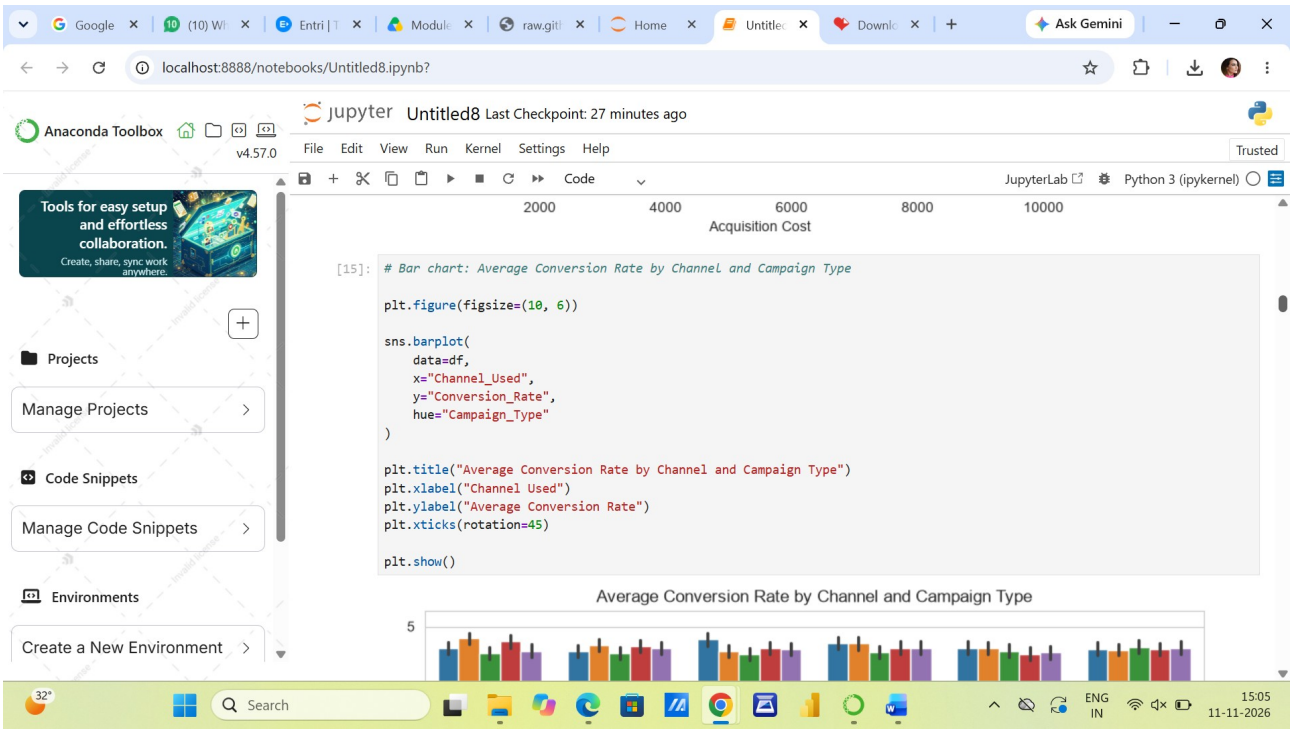


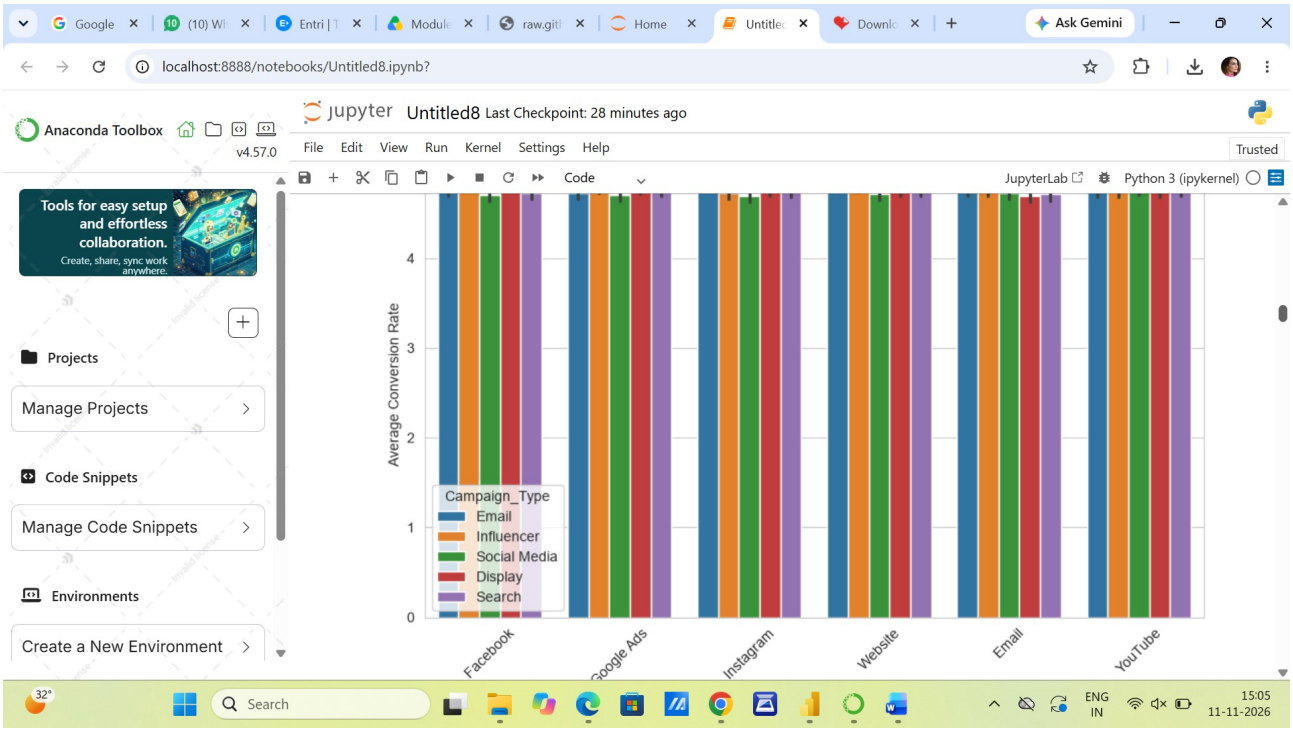


5.2 Bar Chart – Average Conversion Rate by Channel and Campaign Type

The bars compare average conversion rate across marketing channels, with separate colours for campaign types.

```
plt.figure(figsize=(10, 6))
sns.barplot(data=df, x="Channel_Used", y="Conversion_Rate", hue="Campaign_Type")
plt.title("Average Conversion Rate by Channel and Campaign Type")
plt.xlabel("Channel Used")
plt.ylabel("Average Conversion Rate")
plt.xticks(rotation=45)
plt.show()
```

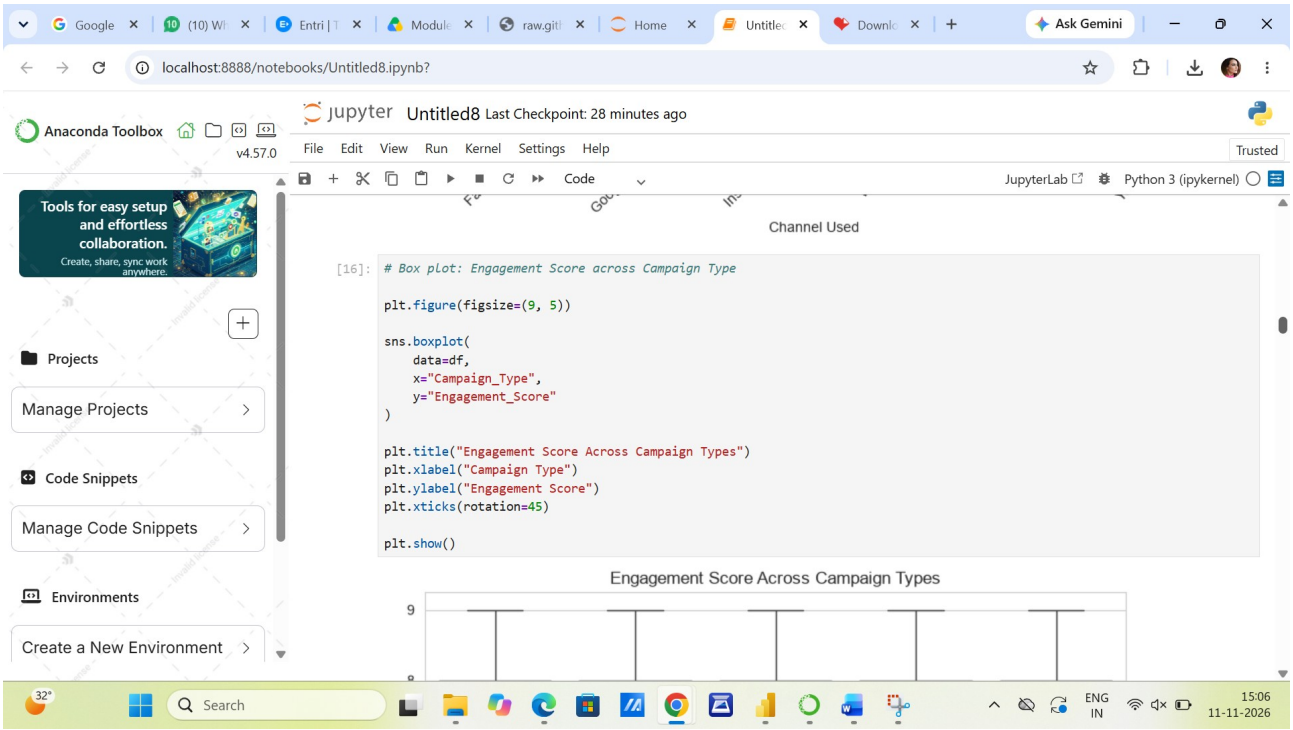


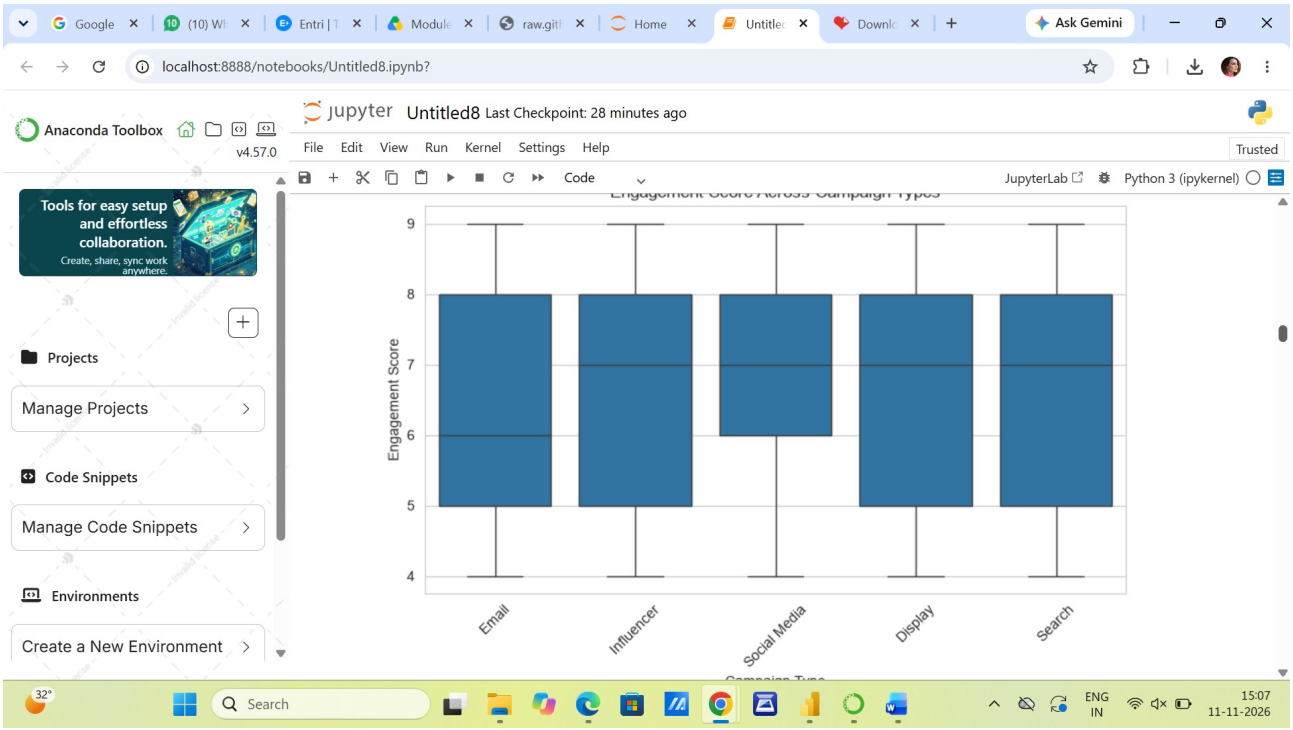


5.3 Box Plot – Engagement Score Across Campaign Type

The box plot compares the distribution of engagement scores across campaign types, including the median and spread.

```
plt.figure(figsize=(9, 5))
sns.boxplot(data=df, x="Campaign_Type", y="Engagement_Score")
plt.title("Engagement Score Across Campaign Types")
plt.xlabel("Campaign Type")
plt.ylabel("Engagement Score")
plt.xticks(rotation=45)
plt.show()
```

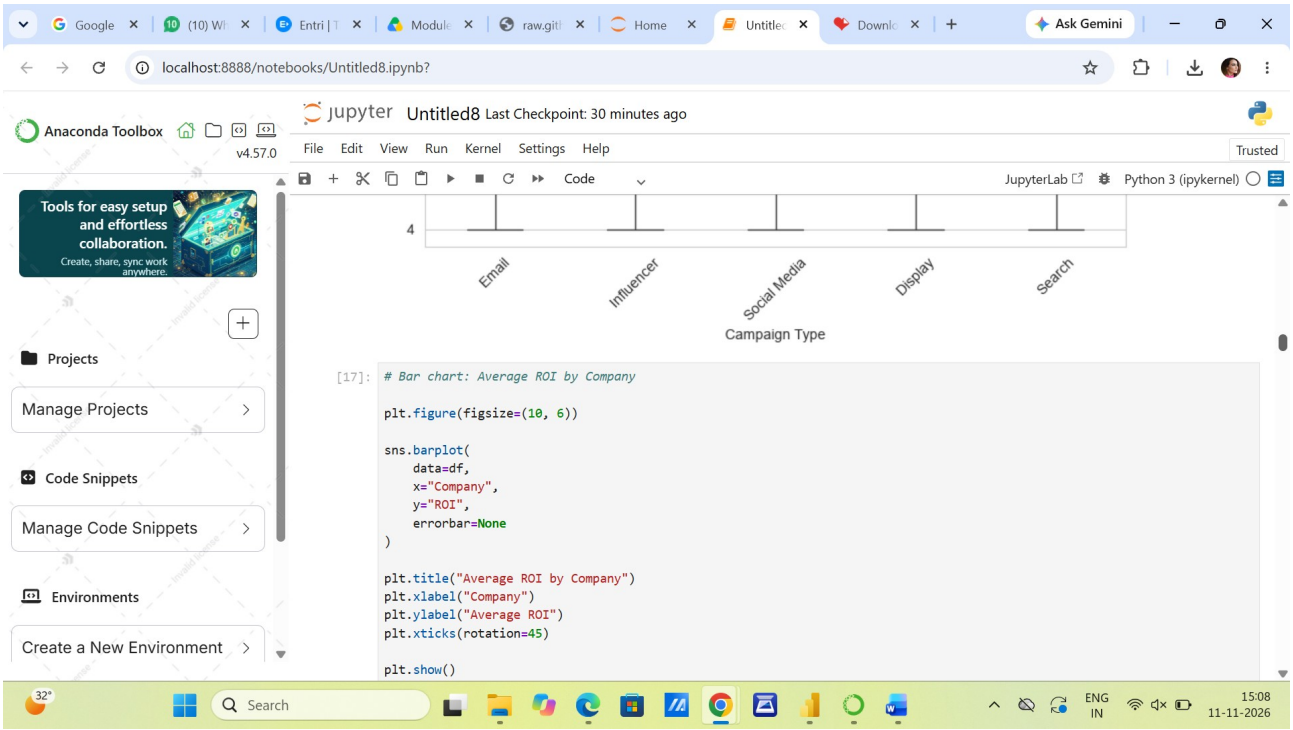


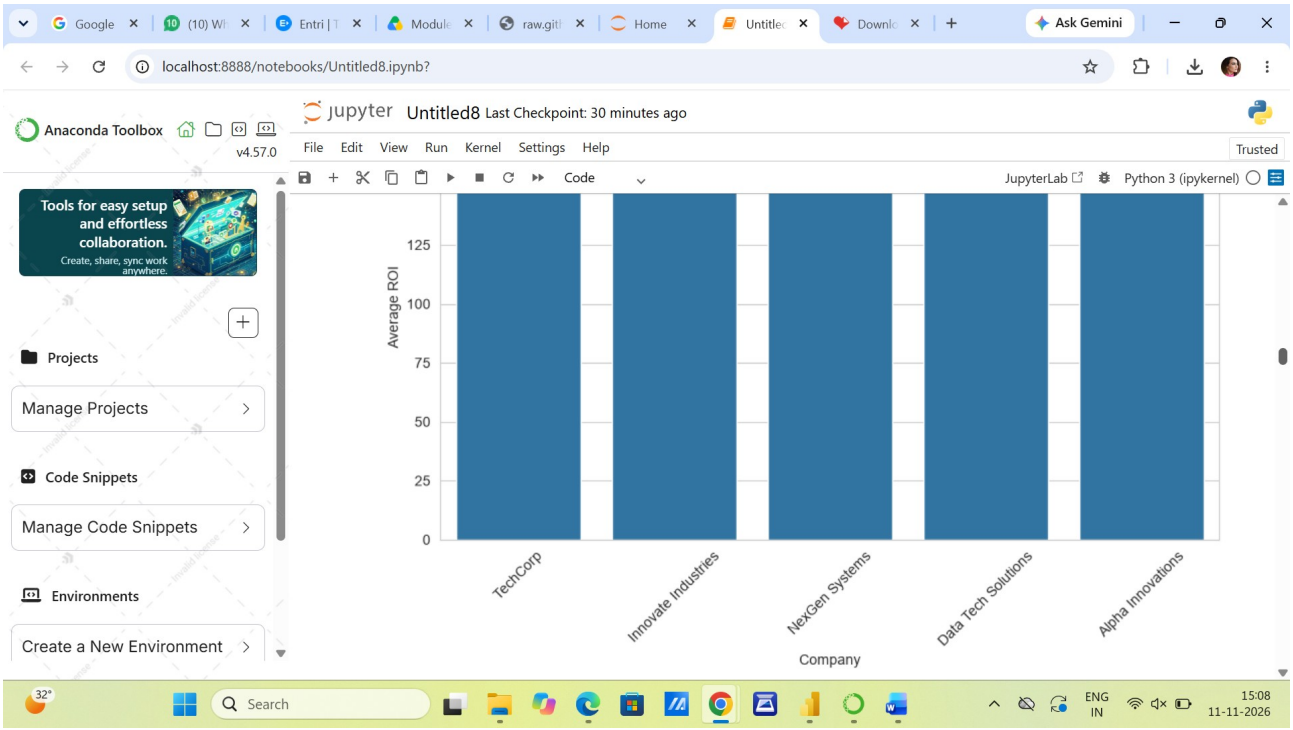


5.4 Bar Chart – Average ROI by Company

Each bar represents the average ROI for a company. This supports comparison of average campaign returns by company.

```
plt.figure(figsize=(10, 6))
sns.barplot(data=df, x="Company", y="ROI", errorbar=None)
plt.title("Average ROI by Company")
plt.xlabel("Company")
plt.ylabel("Average ROI")
plt.xticks(rotation=45)
plt.show()
```

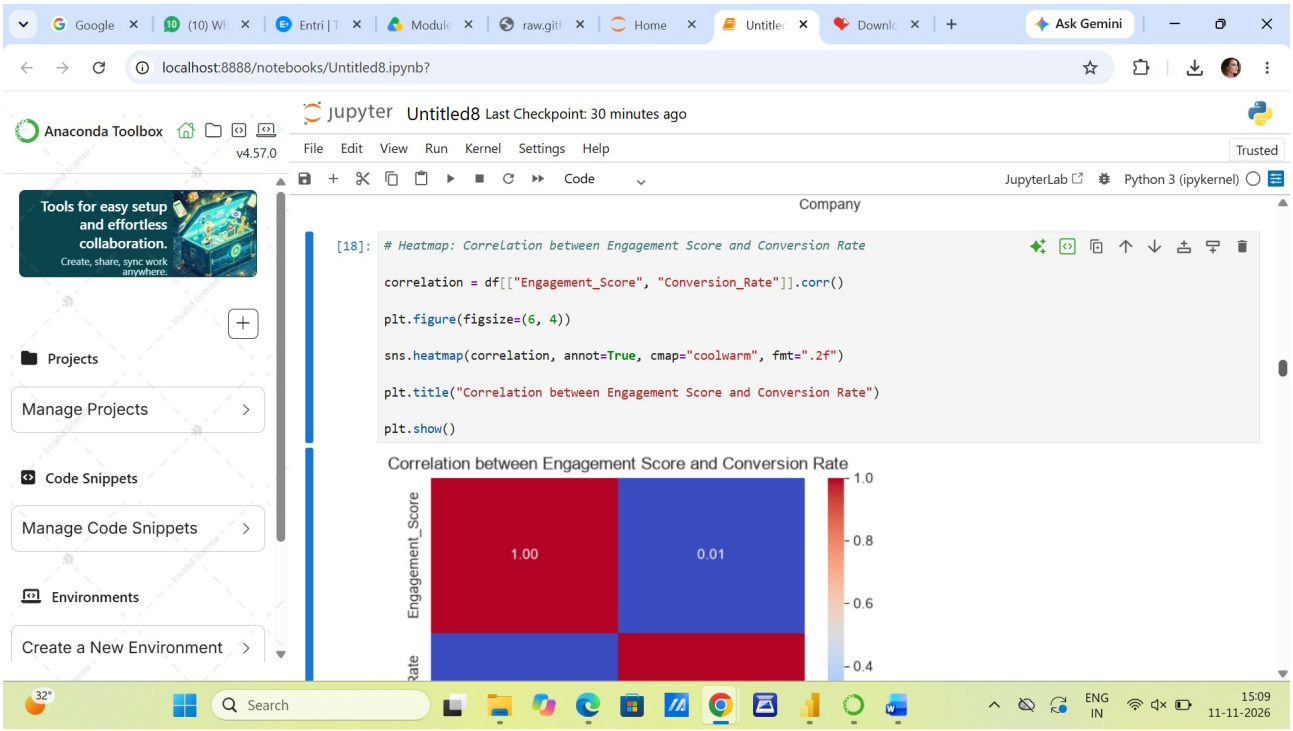




5.5 Heatmap – Correlation Between Engagement Score and Conversion Rate

The heatmap displays the correlation coefficient between engagement score and conversion rate. Values closer to 1 indicate a positive linear relationship, values closer to -1 indicate a negative linear relationship, and values near 0 indicate a weak linear relationship.

```
correlation = df[["Engagement_Score", "Conversion_Rate"]].corr()
plt.figure(figsize=(6, 4))
sns.heatmap(correlation, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation between Engagement Score and Conversion Rate")
plt.show()
```

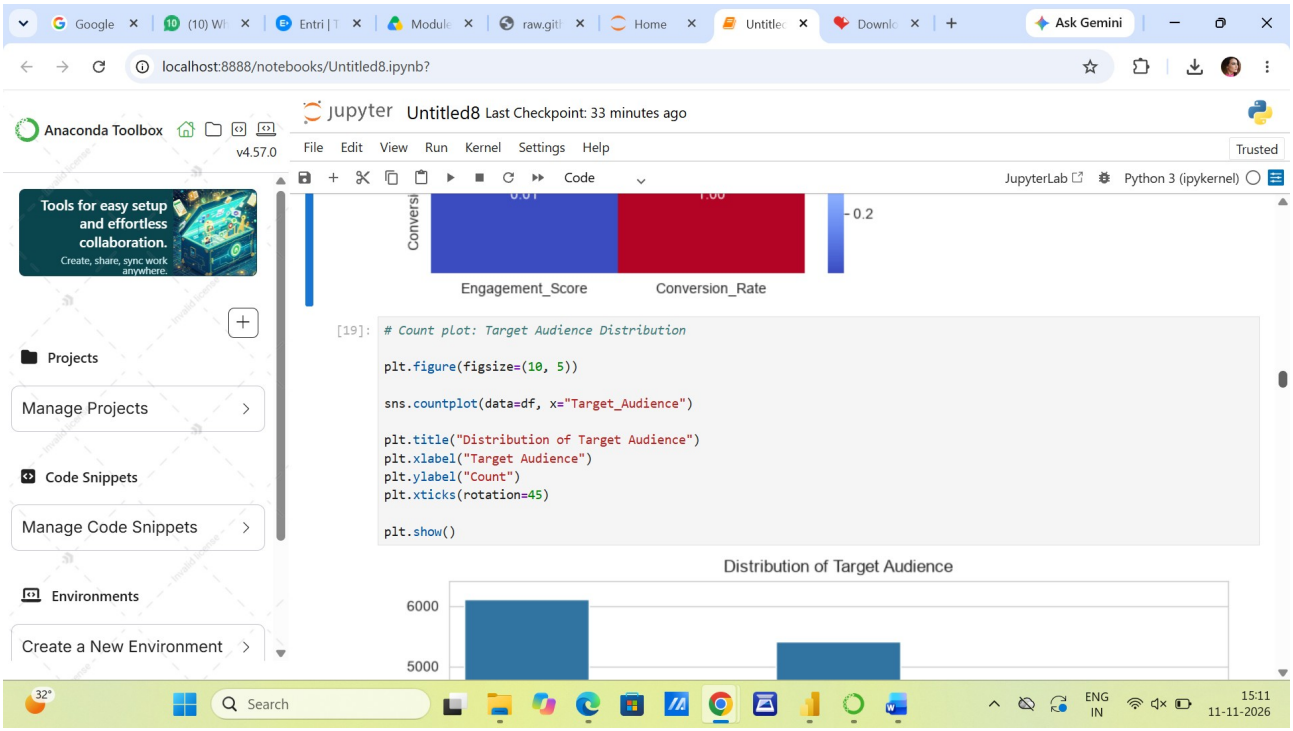


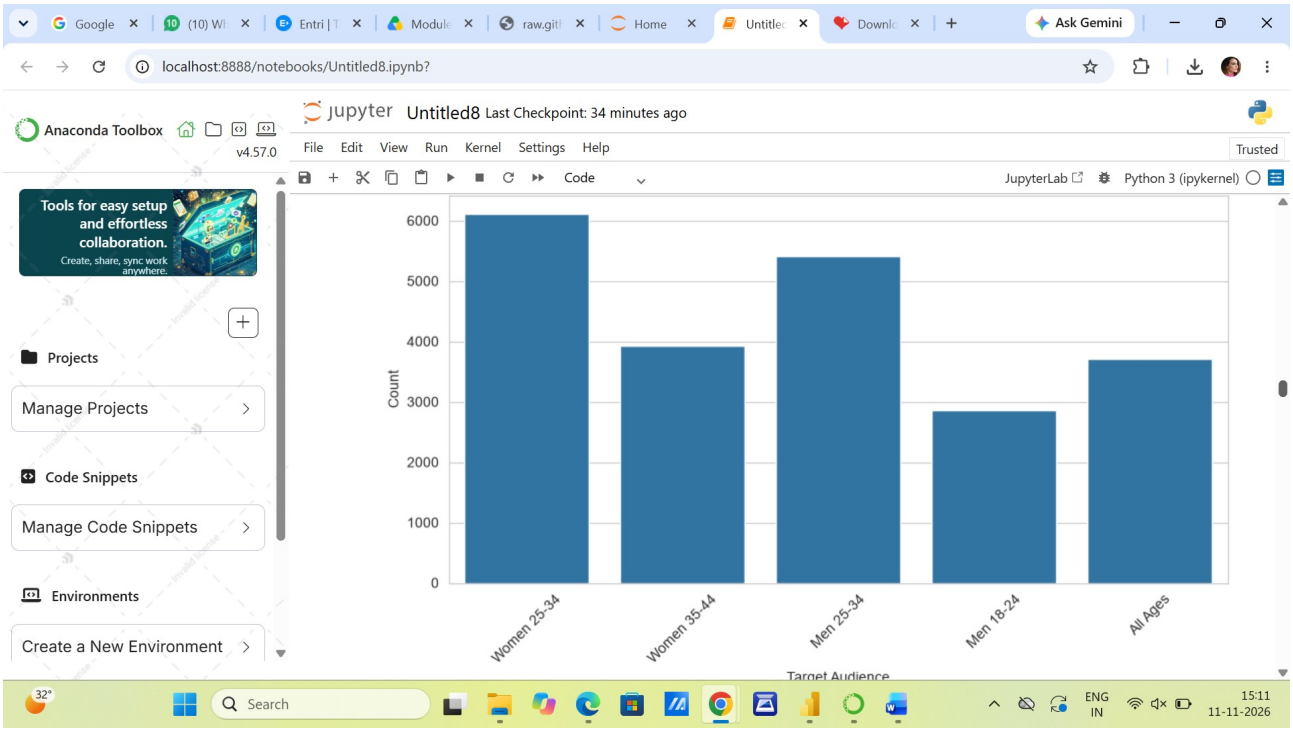


5.6 Count Plot – Target Audience Distribution

The count plot shows the number of campaigns associated with each target audience group.

```
plt.figure(figsize=(10, 5))
sns.countplot(data=df, x="Target_Audience")
plt.title("Distribution of Target Audience")
plt.xlabel("Target Audience")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()
```




Anaconda Toolbox
v4.57.0
Tools for easy setup and effortless collaboration.
Create, share, sync work anywhere.
Projects
Manage Projects
Code Snippets
Manage Code Snippets
Environments
Create a New Environment
jupyter
Untitled8 Last Checkpoint: 34 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
Count
6000
5000
4000
3000
2000
1000
0
Women 25-34
Women 35-44
Men 25-34
Men 18-24
All Ages
Target Audience
Search
ENG
IN
15:11
11-11-2026

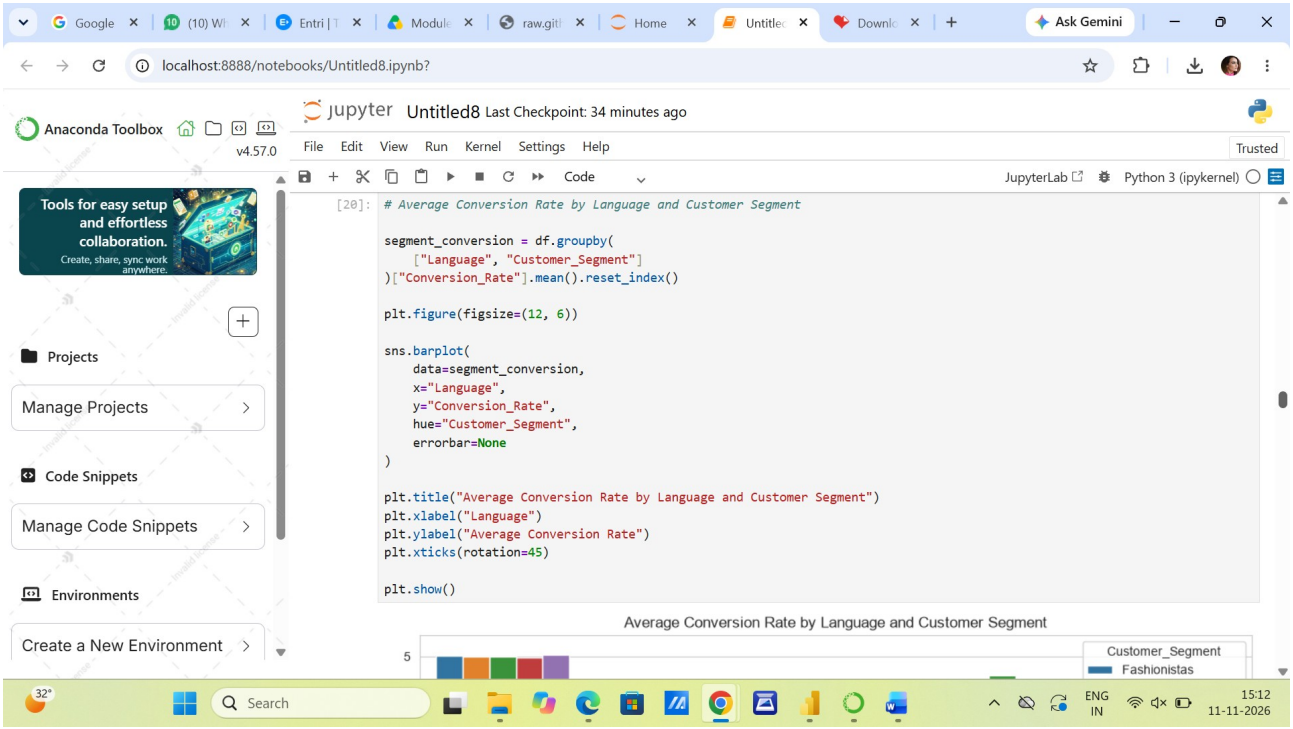
5.7 Highest Conversion Rate by Customer Segment for Each Language

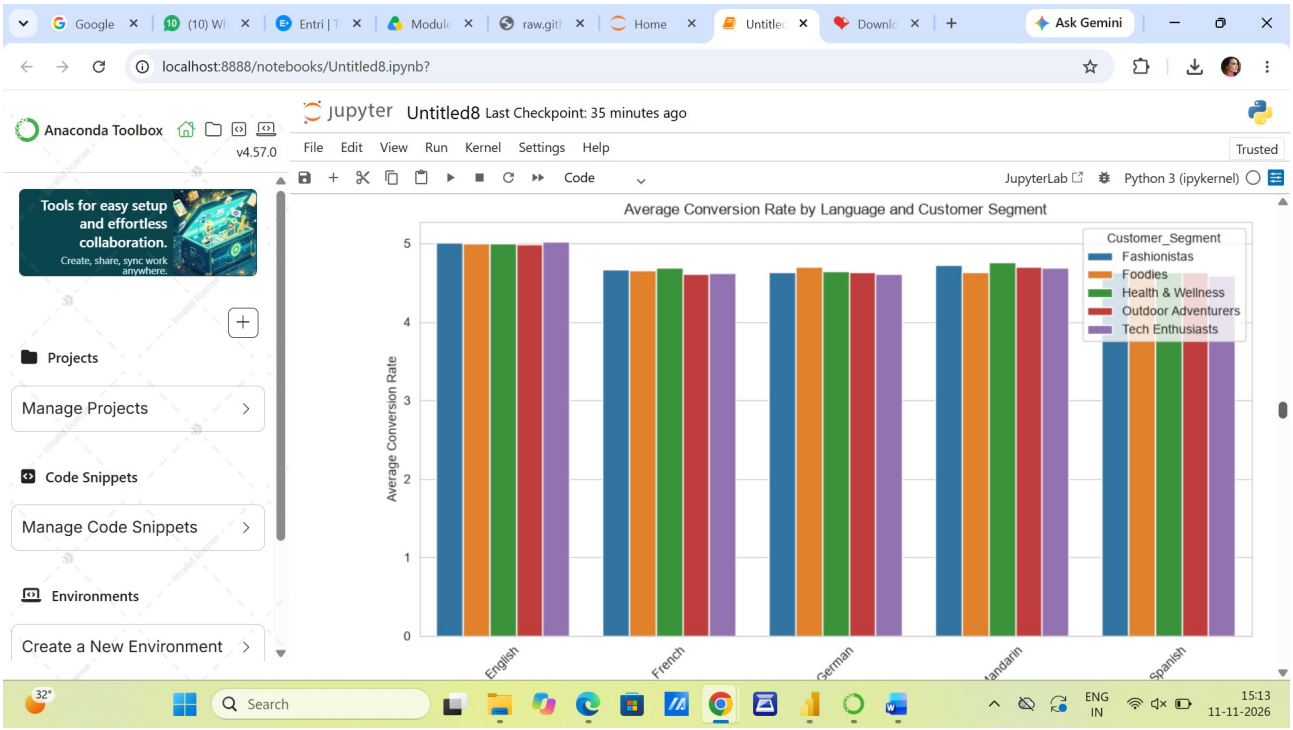
The grouped bar chart compares average conversion rates for customer segments within each language. The printed table identifies the highest-converting customer segment for each language.

```
segment_conversion =
    df.groupby( ["Language",
    "Customer_Segment"]
)["Conversion_Rate"].mean().reset_index()

plt.figure(figsize=(12, 6))
sns.barplot(
    data=segment_conversion,
    x="Language",
    y="Conversion_Rate",
    hue="Customer_Segment",
    errorbar=None
)
plt.title("Average Conversion Rate by Language and Customer Segment")
plt.xlabel("Language")
plt.ylabel("Average Conversion Rate")
plt.xticks(rotation=45)
plt.show()

highest_segments = segment_conversion.loc[
    segment_conversion.groupby("Language")["Conversion_Rate"].idxmax()
]
print(highest_segments)
```

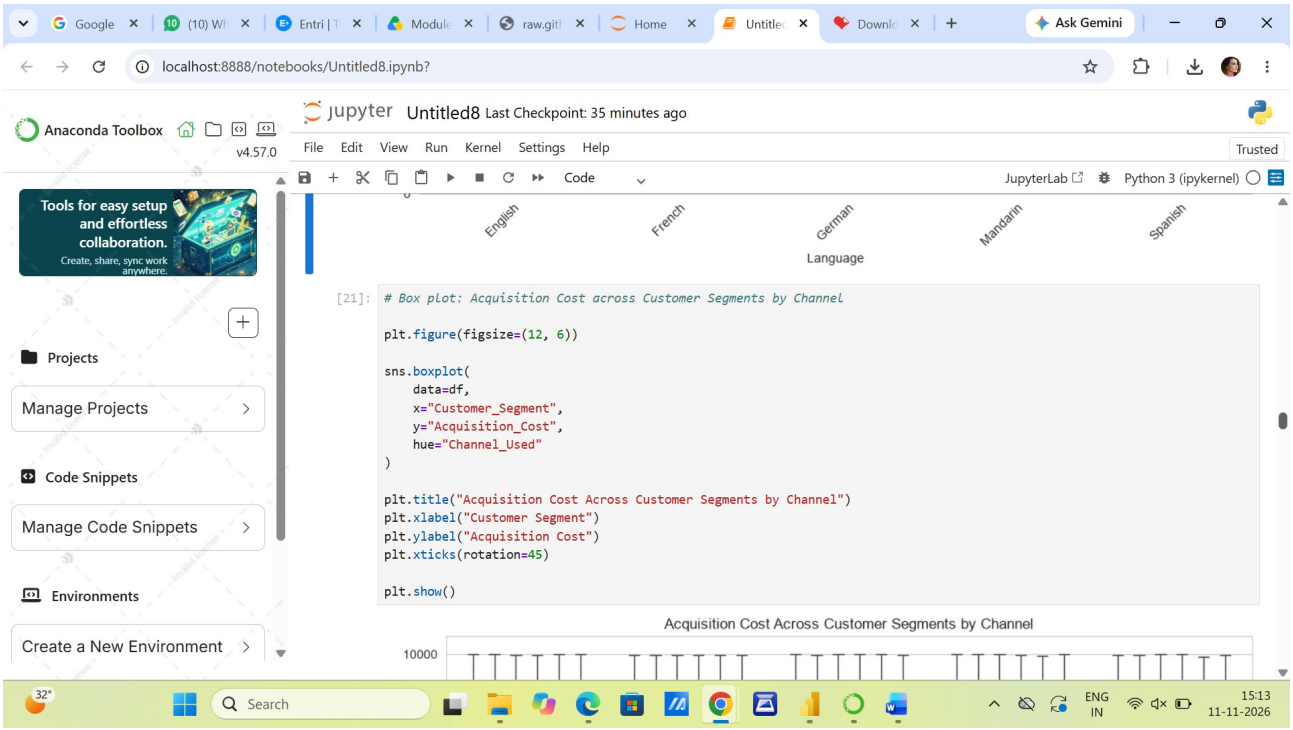


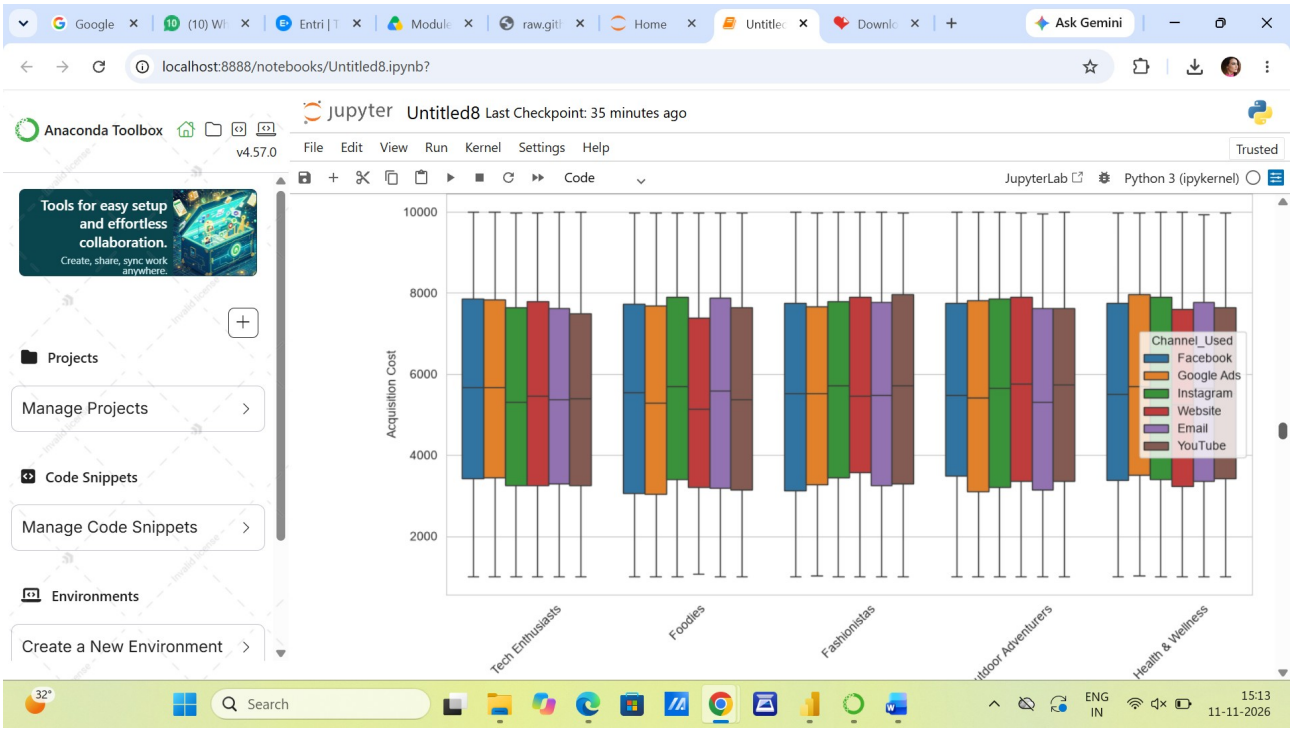


5.8 Box Plot – Acquisition Cost Across Customer Segments by Channel

The box plot compares acquisition cost distributions across customer segments, with channels distinguished by colour.

```
plt.figure(figsize=(12, 6))
sns.boxplot(
    data=df,
    x="Customer_Segment",
    y="Acquisition_Cost",
    hue="Channel_Used"
)
plt.title("Acquisition Cost Across Customer Segments by Channel")
plt.xlabel("Customer Segment")
plt.ylabel("Acquisition Cost")
plt.xticks(rotation=45)
plt.show()
```

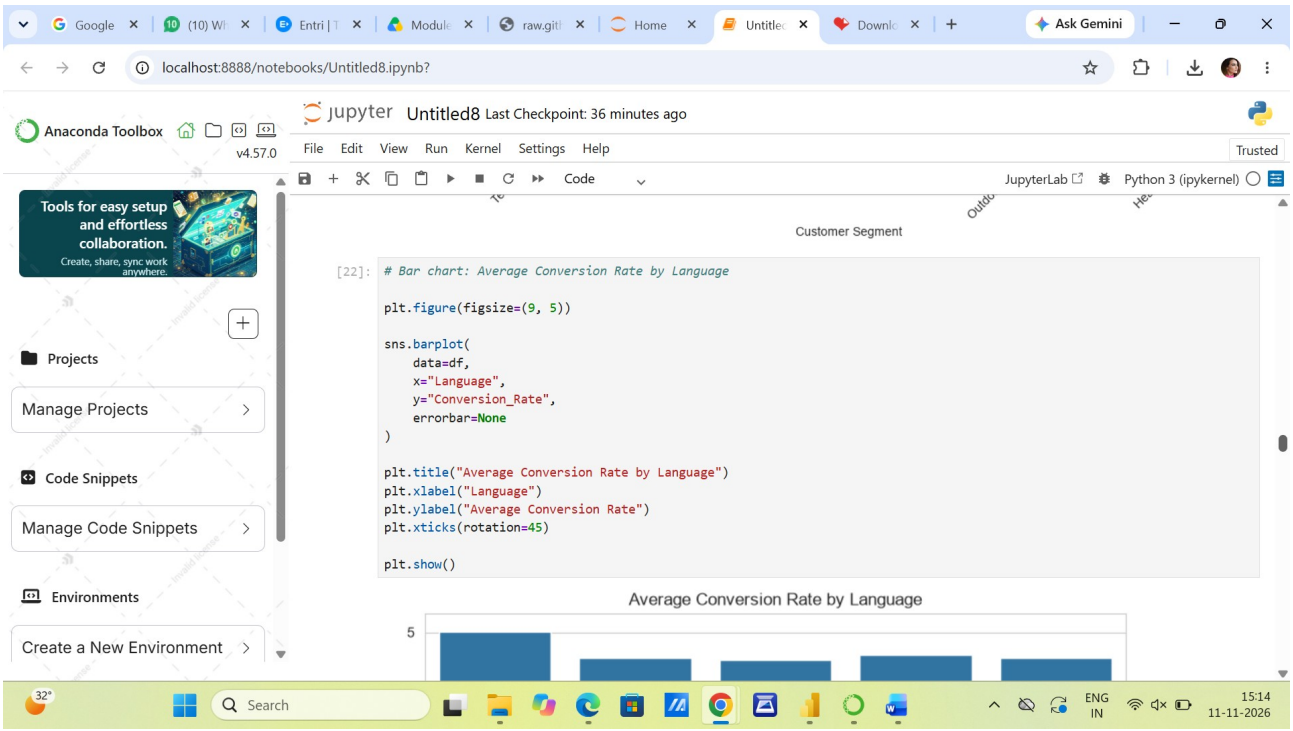


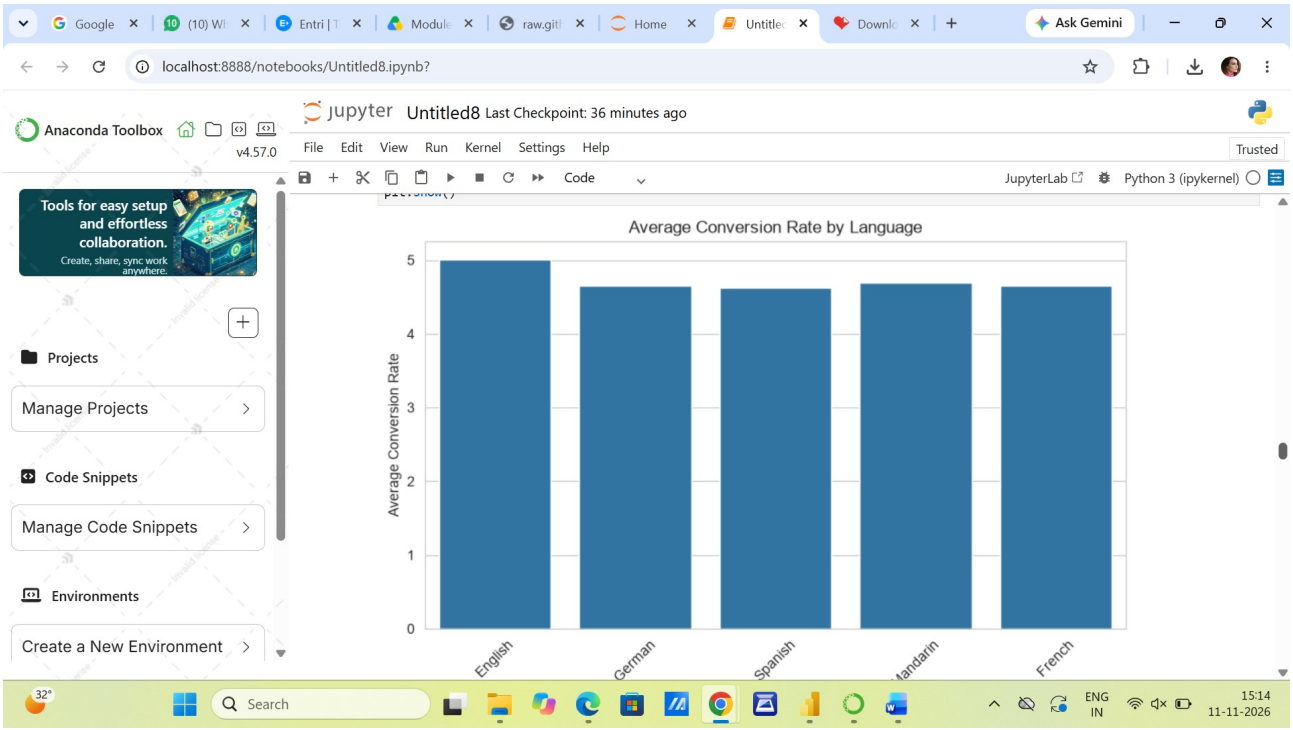


5.9 Bar Chart – Average Conversion Rate by Language

The bar chart compares the average conversion rate for each language.

```
plt.figure(figsize=(9, 5))
sns.barplot(data=df, x="Language", y="Conversion_Rate", errorbar=None)
plt.title("Average Conversion Rate by Language")
plt.xlabel("Language")
plt.ylabel("Average Conversion Rate")
plt.xticks(rotation=45)
plt.show()
```



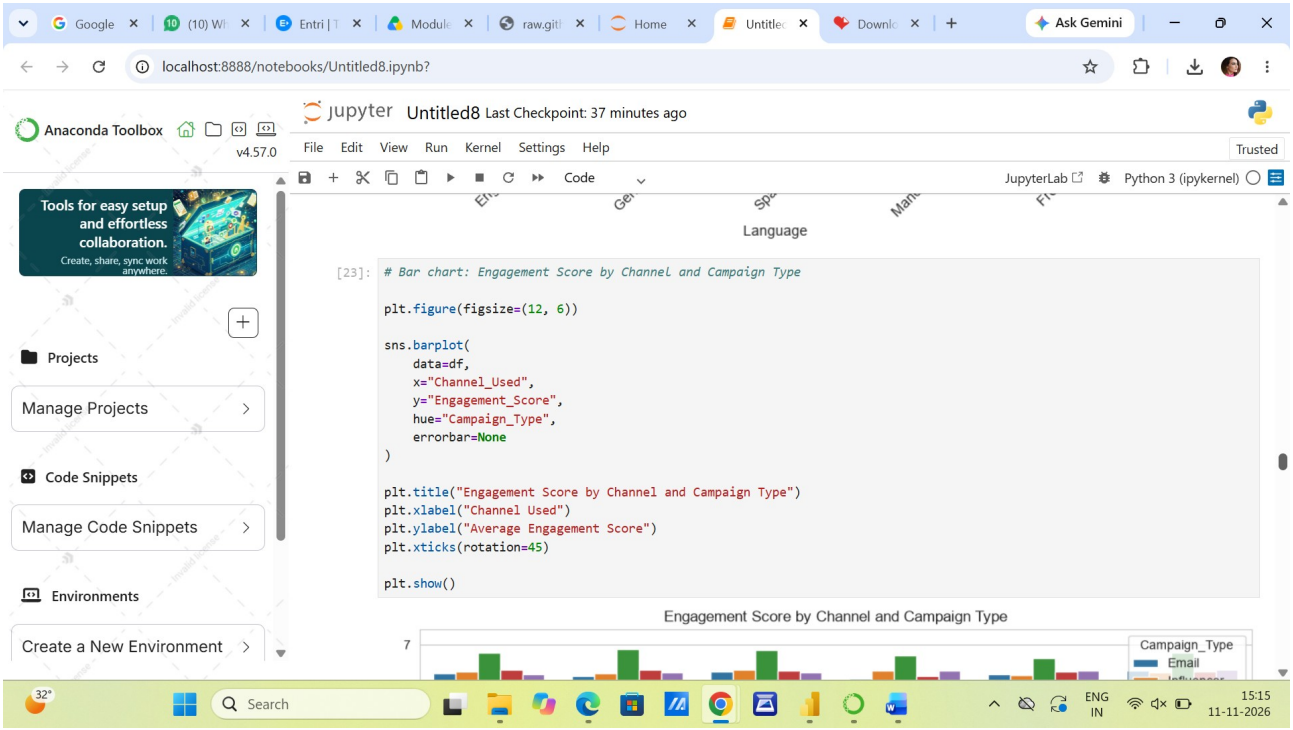


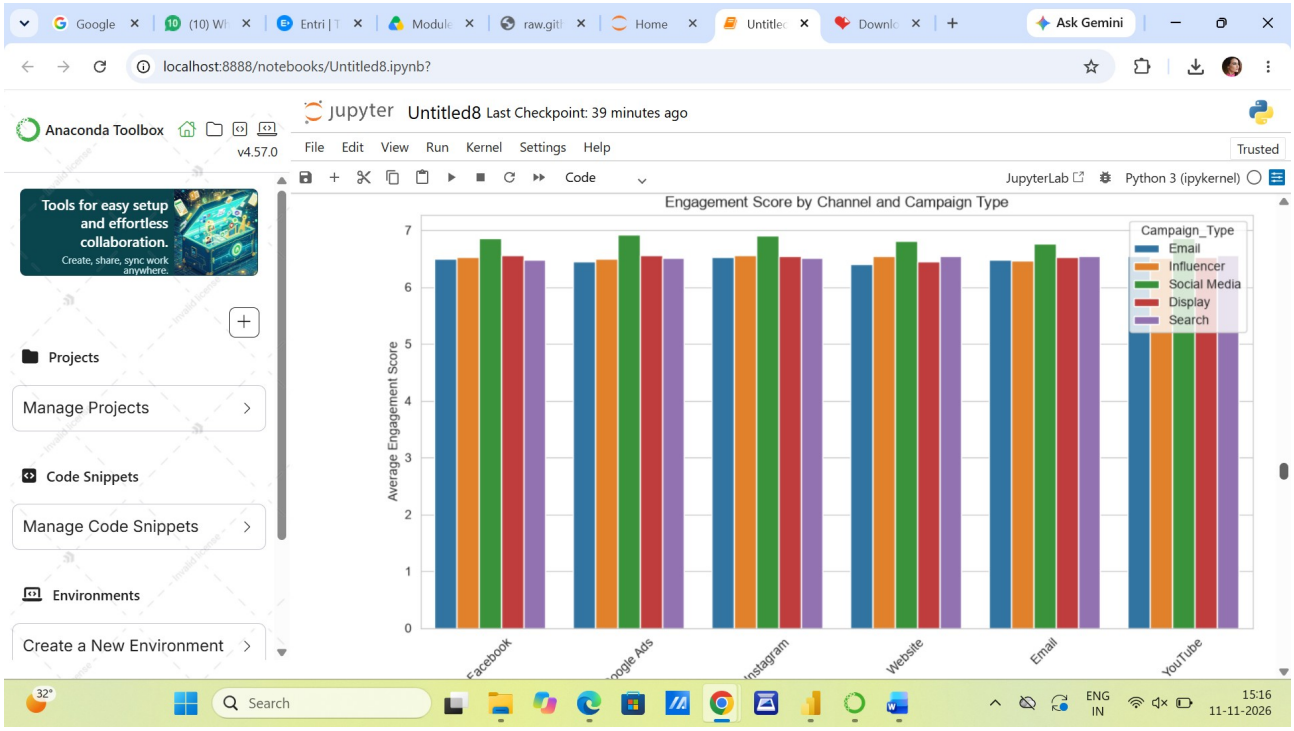
5.10 Bar Chart – Engagement Score by Channel and Campaign Type

The chart compares average engagement scores across channels, separated by campaign type.

```
plt.figure(figsize=(12, 6))
sns.barplot(
    data=df,
    x="Channel_Used",
    y="Engagement_Score",
    hue="Campaign_Type",
    errorbar=None
)
plt.title("Engagement Score by Channel and Campaign Type")
plt.xlabel("Channel Used")
plt.ylabel("Average Engagement Score")
plt.xticks(rotation=45)

plt.show()
```

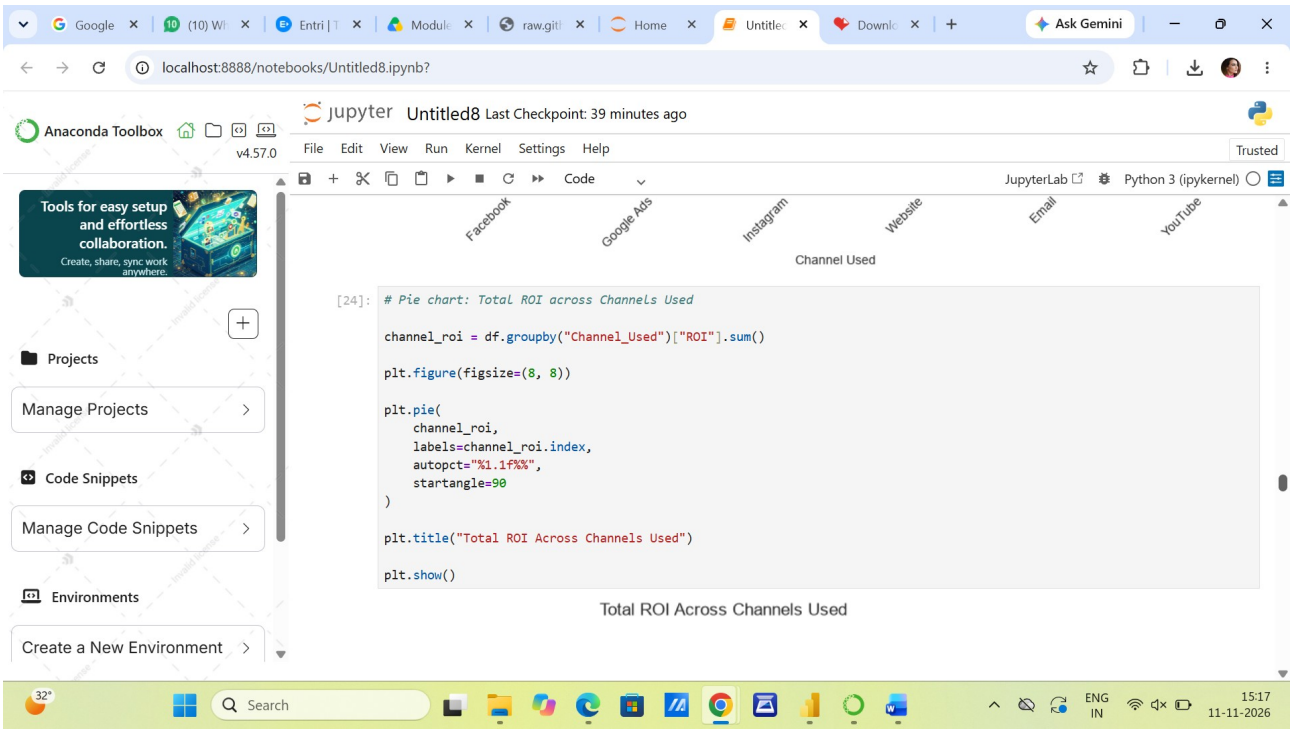


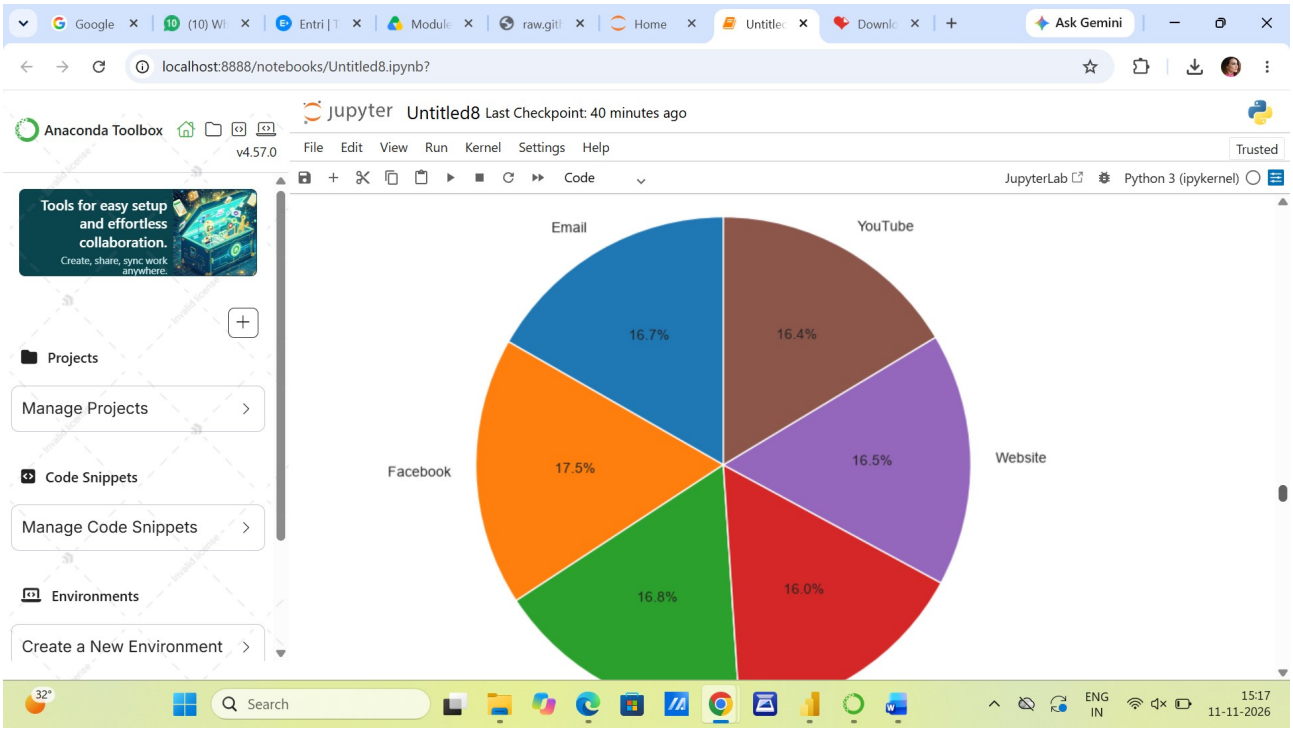


5.11 Pie Chart – Total ROI Across Channels Used

Each pie slice represents a channel's share of the total ROI across all channels.

```
channel_roi = df.groupby("Channel_Used")["ROI"].sum()
plt.figure(figsize=(8, 8))
plt.pie(
    channel_roi,
    labels=channel_roi.index,
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Total ROI Across Channels Used")
plt.show()
```

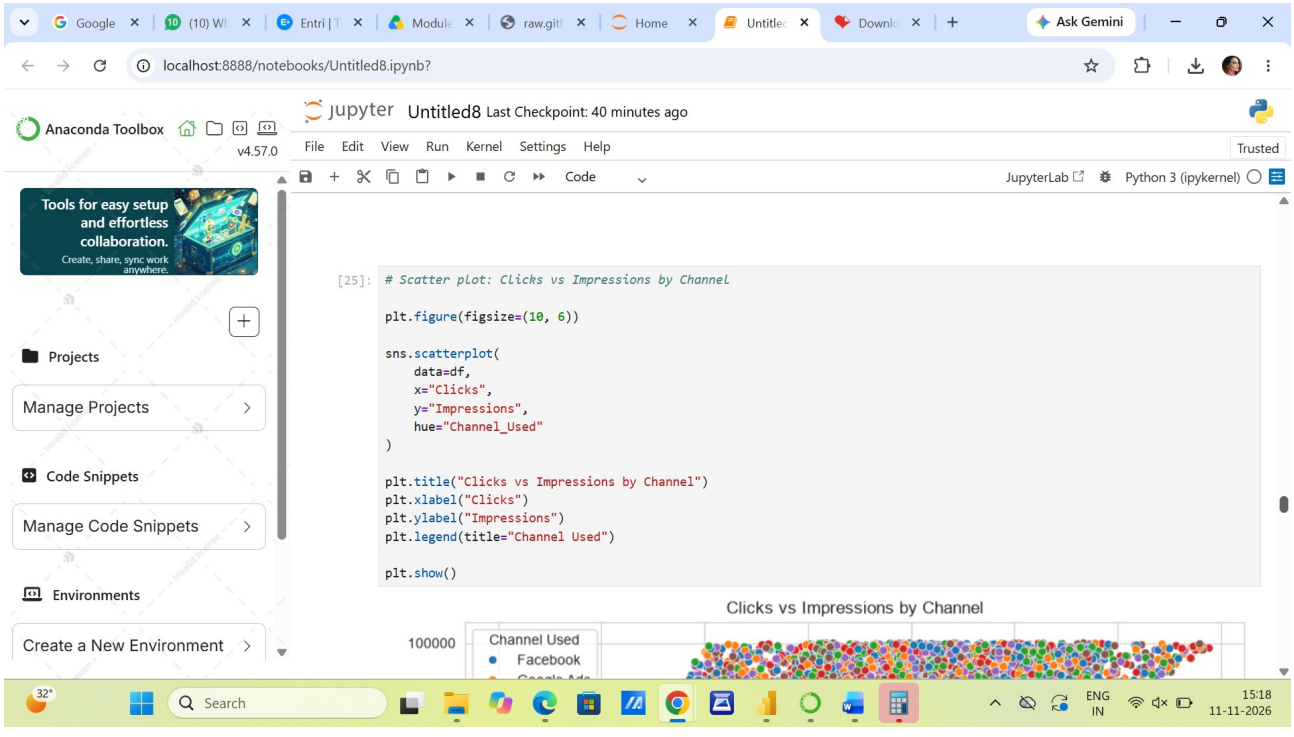


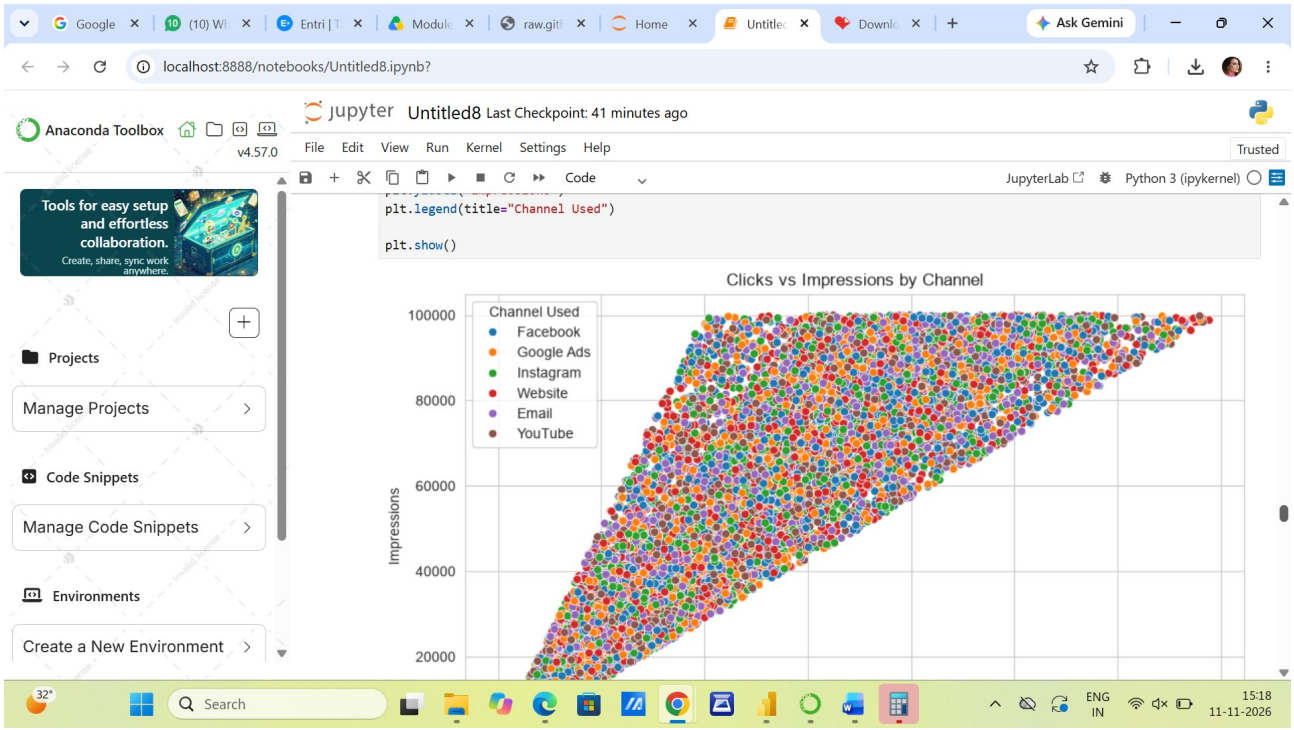


5.12 Scatter Plot – Clicks vs Impressions by Channel

The scatter plot compares clicks and impressions. Different colours distinguish the marketing channels.

```
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df,
    x="Clicks",
    y="Impressions",
    hue="Channel_Used"
)
plt.title("Clicks vs Impressions by Channel")
plt.xlabel("Clicks")
plt.ylabel("Impressions")
plt.legend(title="Channel Used")
plt.show()
```

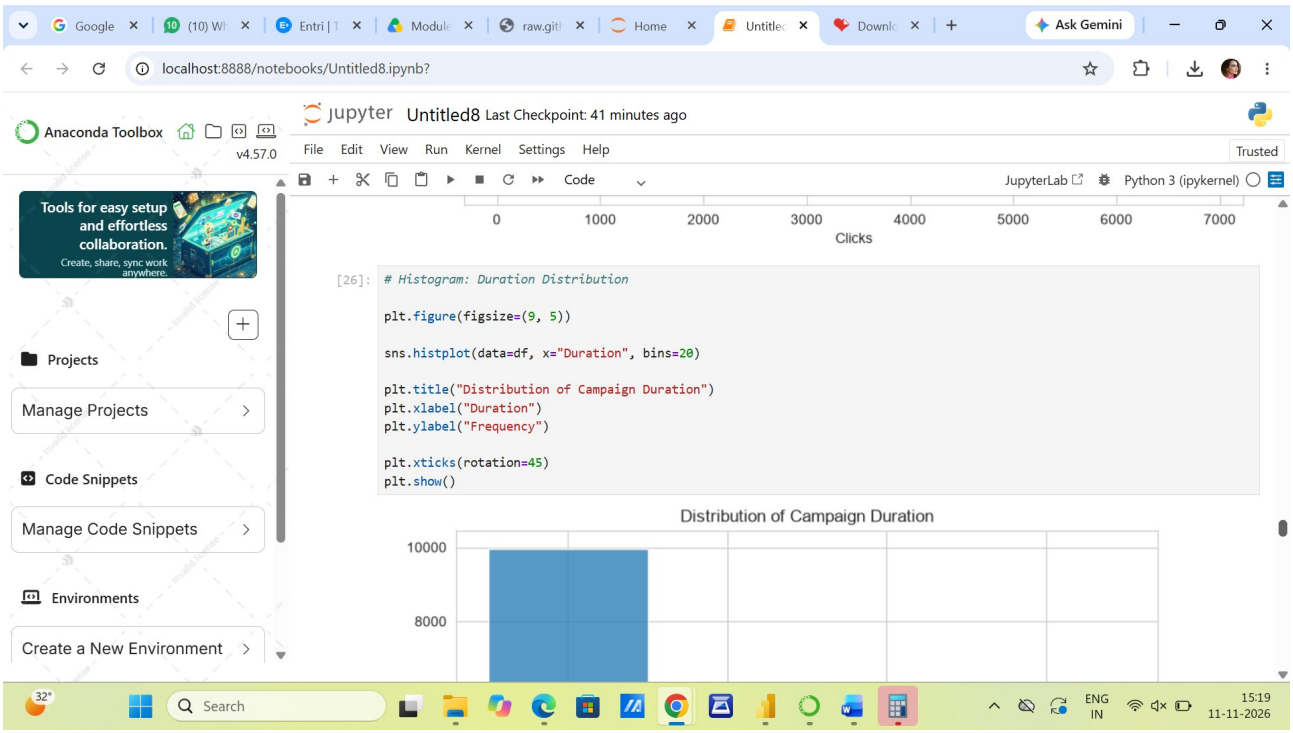


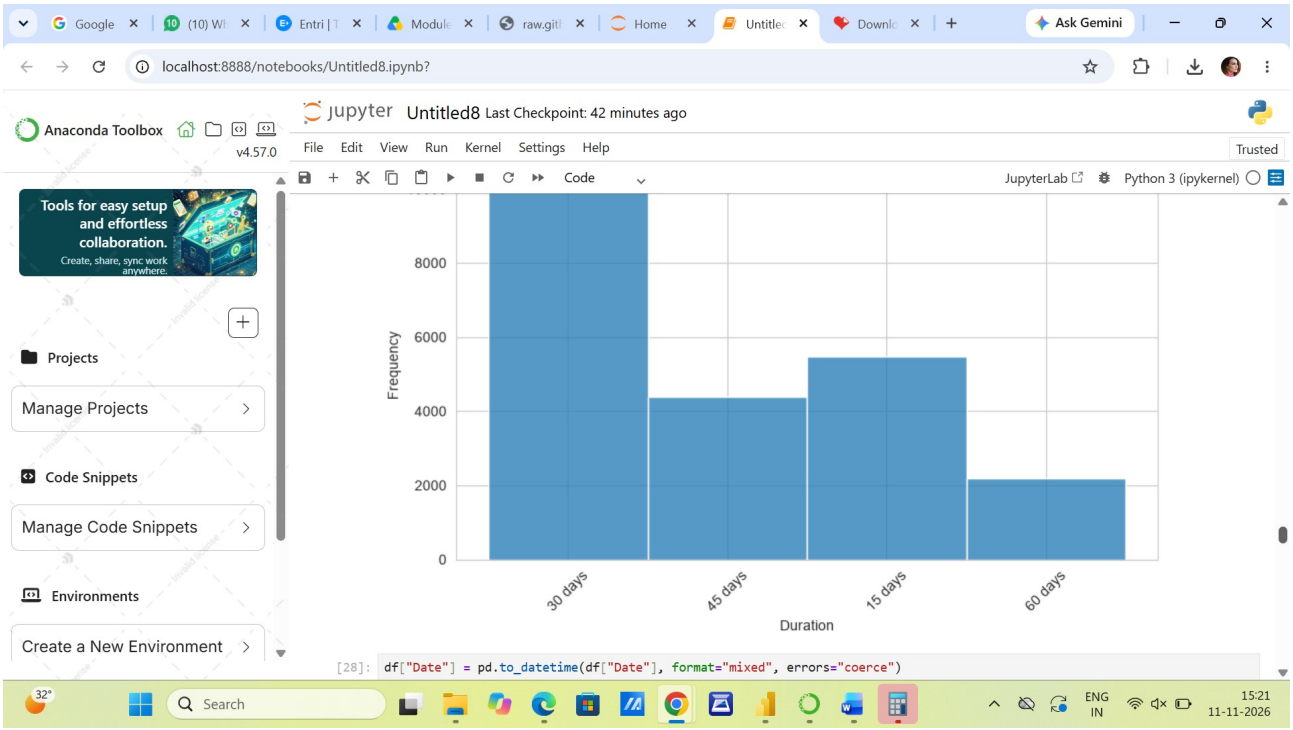


5.13 Histogram – Duration Distribution

The histogram displays the frequency distribution of campaign duration values.

```
plt.figure(figsize=(9, 5))
sns.histplot(data=df, x="Duration", bins=20)
plt.title("Distribution of Campaign Duration")
plt.xlabel("Duration")
plt.ylabel("Frequency")
plt.xticks(rotation=45)
plt.show()
```





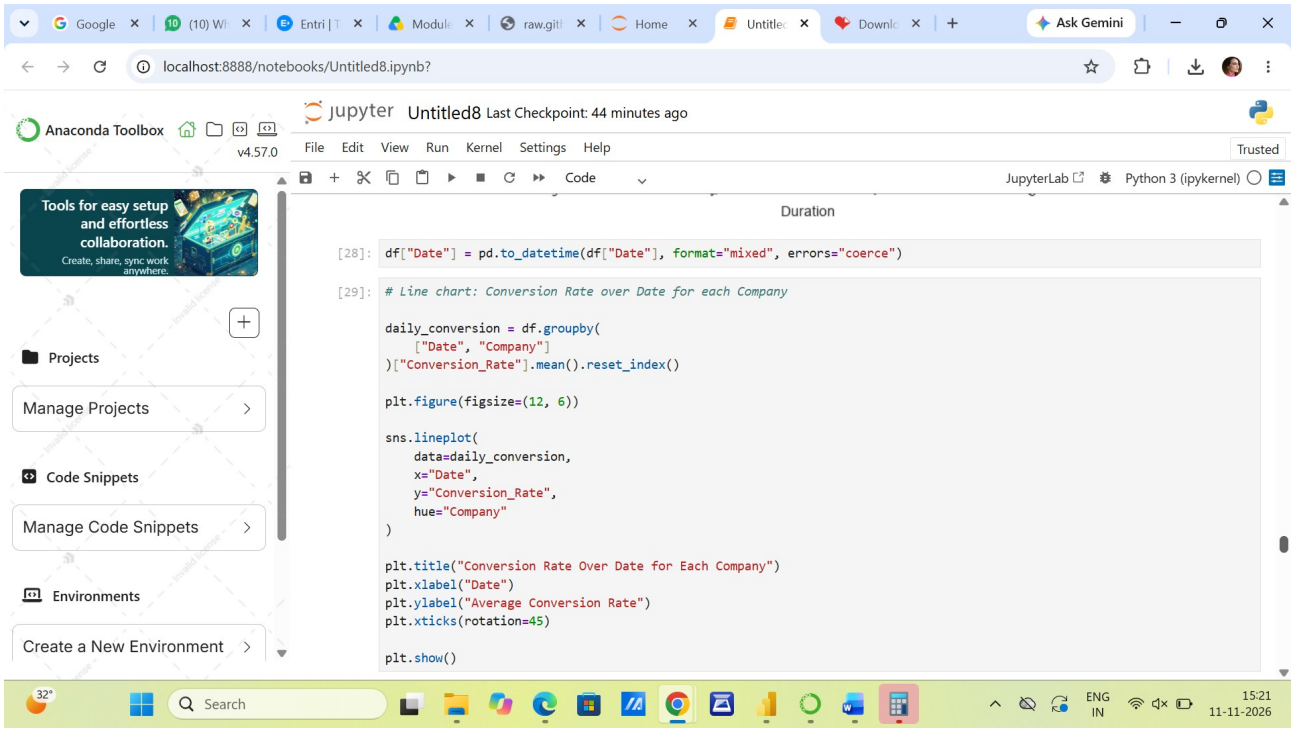
5.14 Line Chart – Conversion Rate Over Date for Each Company

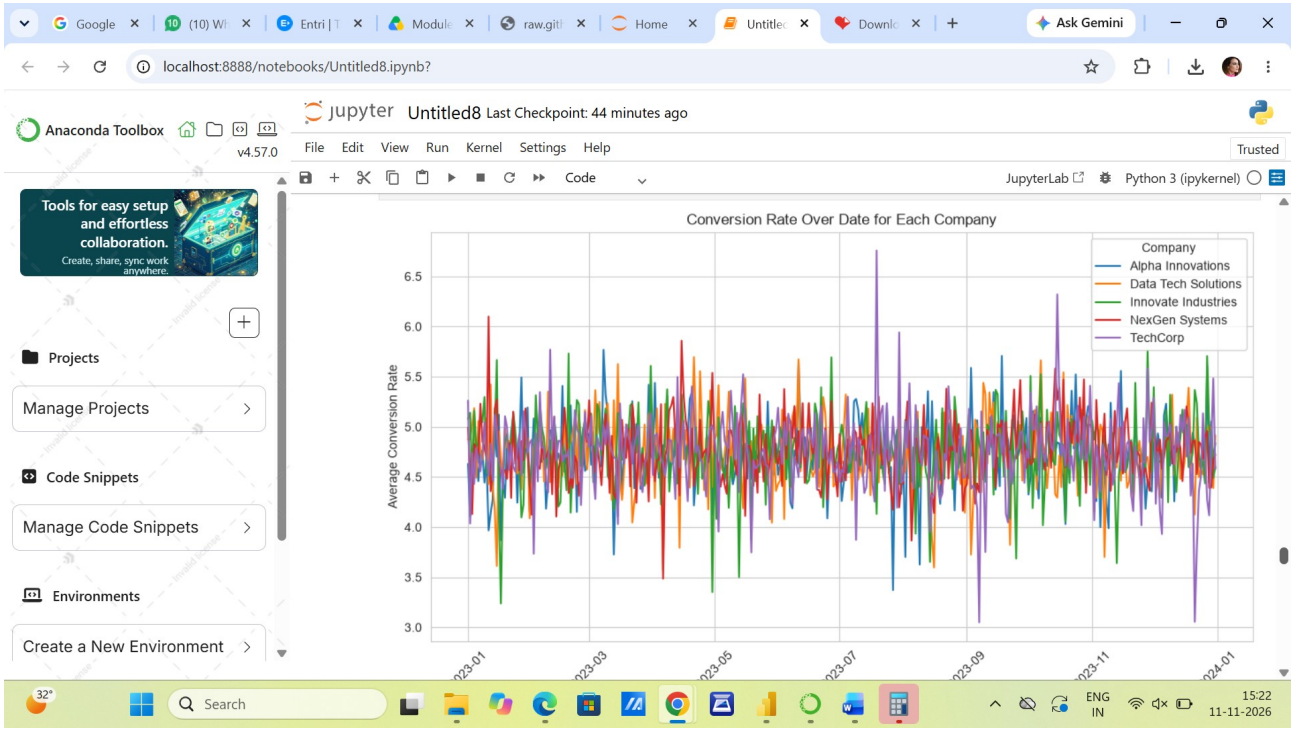
The line chart shows the average conversion rate over time, with a separate line for each company. The Date column is converted to datetime format before plotting.

```
df["Date"] = pd.to_datetime(df["Date"], format="mixed", errors="coerce")

daily_conversion =
    df.groupby( ["Date",
    "Company"]
)["Conversion_Rate"].mean().reset_index()

plt.figure(figsize=(12, 6))
sns.lineplot(
    data=daily_conversion,
    x="Date",
    y="Conversion_Rate",
    hue="Company"
)
plt.title("Conversion Rate Over Date for Each Company")
plt.xlabel("Date")
plt.ylabel("Average Conversion Rate")
plt.xticks(rotation=45)
plt.show()
```





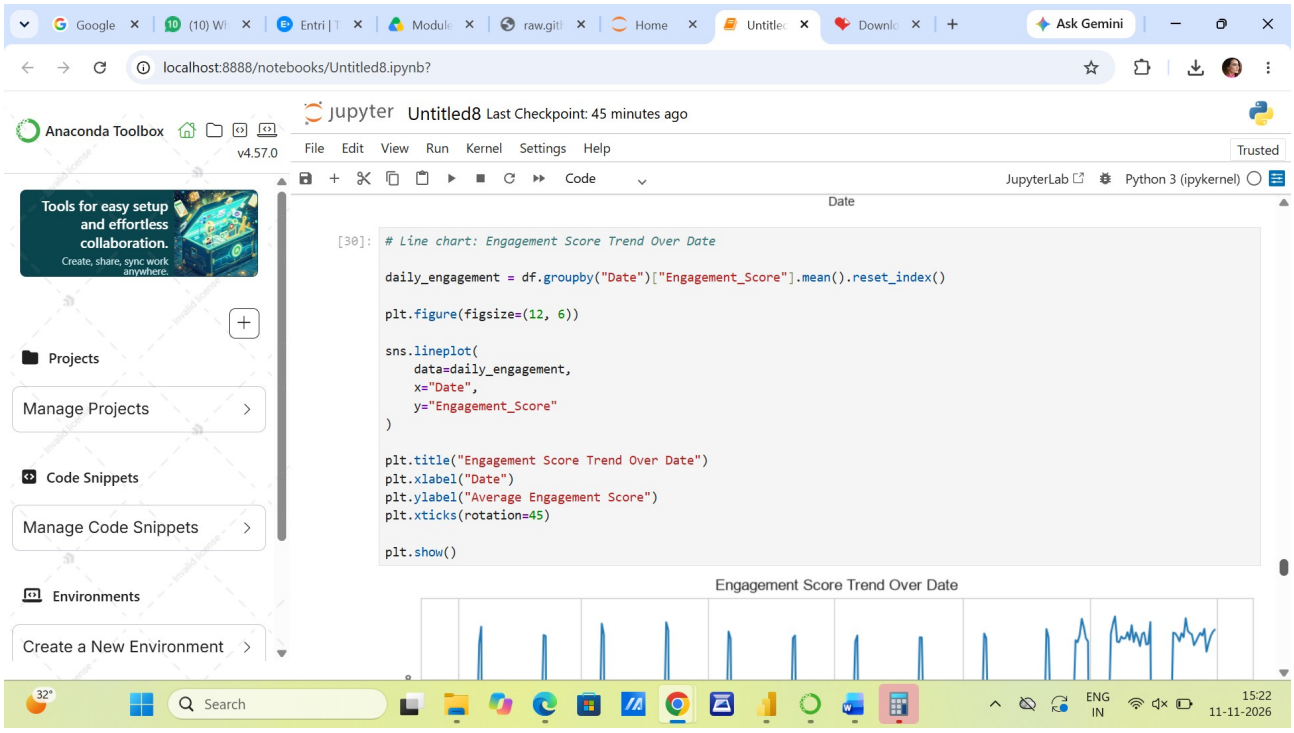
5.15 Line Chart – Engagement Score Trend Over Date

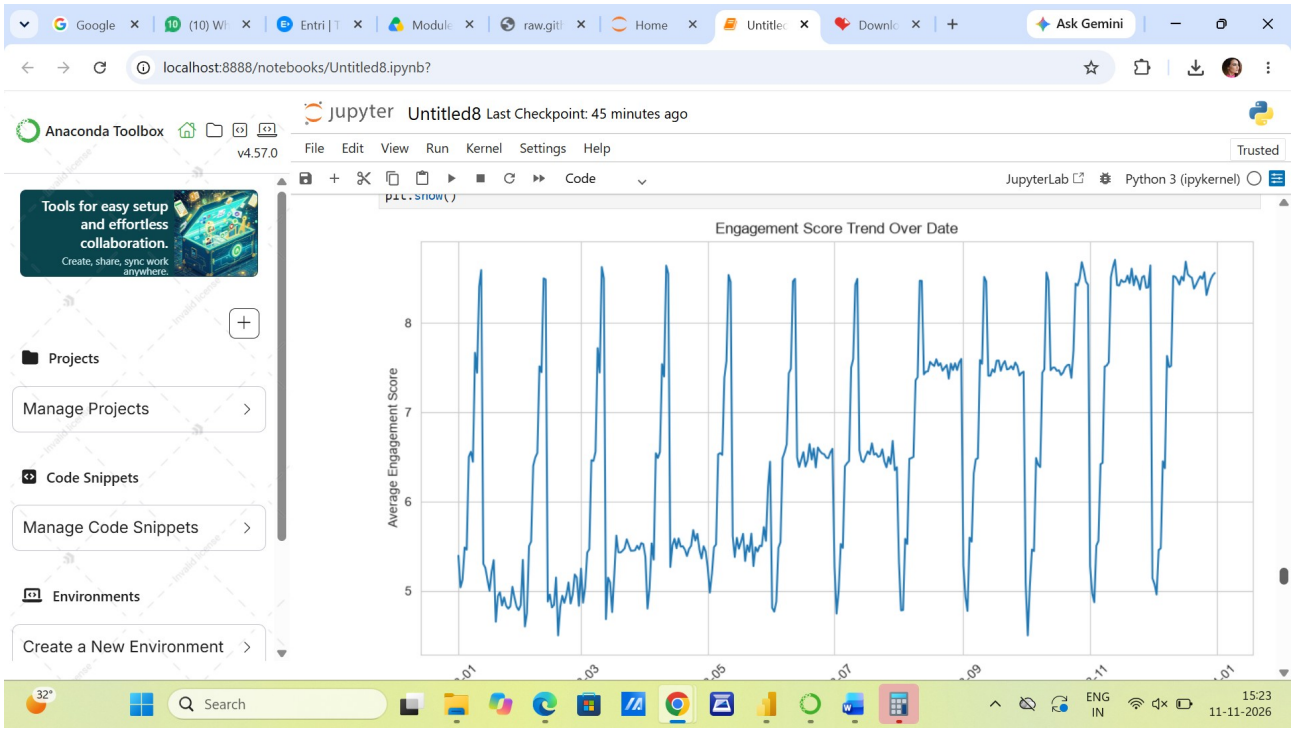
The line chart displays the average engagement score trend over the date range.

```
daily_engagement = df.groupby("Date")["Engagement_Score"].mean().reset_index()

plt.figure(figsize=(12, 6))
sns.lineplot(
    data=daily_engagement,
```

```
    x="Date",
    y="Engagement_Score"
)
plt.title("Engagement Score Trend Over Date")
plt.xlabel("Date")
plt.ylabel("Average Engagement Score")
plt.xticks(rotation=45)
plt.show()
```





5.16 Bar Chart – Average Acquisition Cost by Location

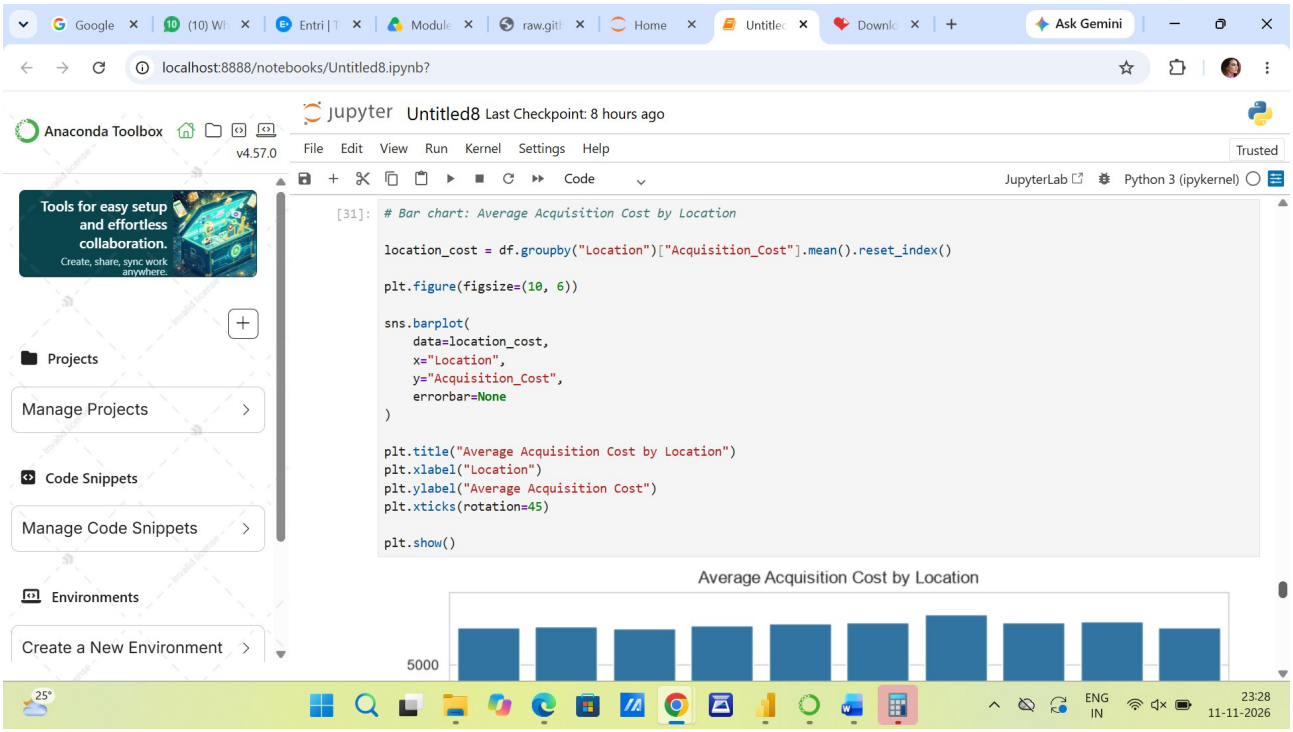
The chart compares average acquisition cost by location. The printed result identifies the location with the highest average acquisition cost.

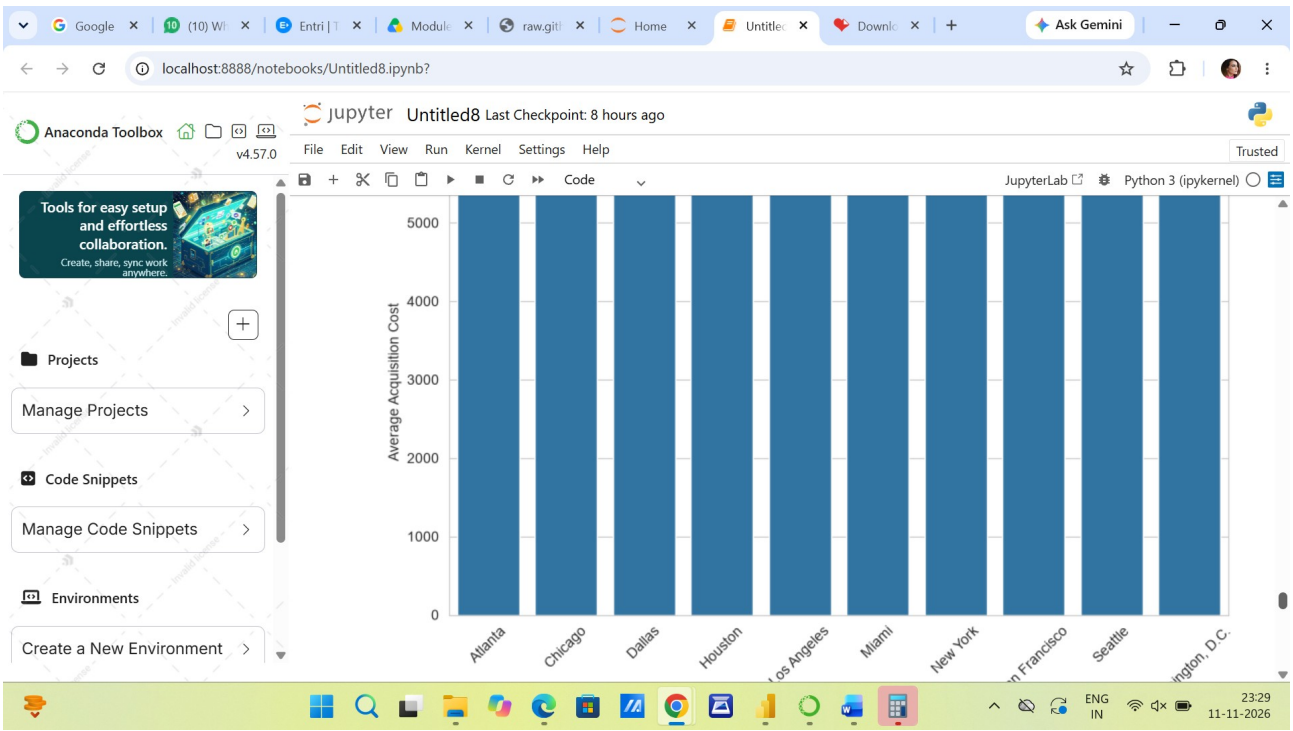
```
location_cost  =  df.groupby("Location")["Acquisition_Cost"].mean().reset_index()

plt.figure(figsize=(10, 6))
sns.barplot(
    data=location_cost,
    x="Location",
    y="Acquisition_Cost",
```

```
    errorbar=None
)
plt.title("Average Acquisition Cost by Location")
plt.xlabel("Location")
plt.ylabel("Average Acquisition Cost")
plt.xticks(rotation=45)
plt.show()

print(location_cost.sort_values("Acquisition_Cost",  ascending=False).head(1))
```



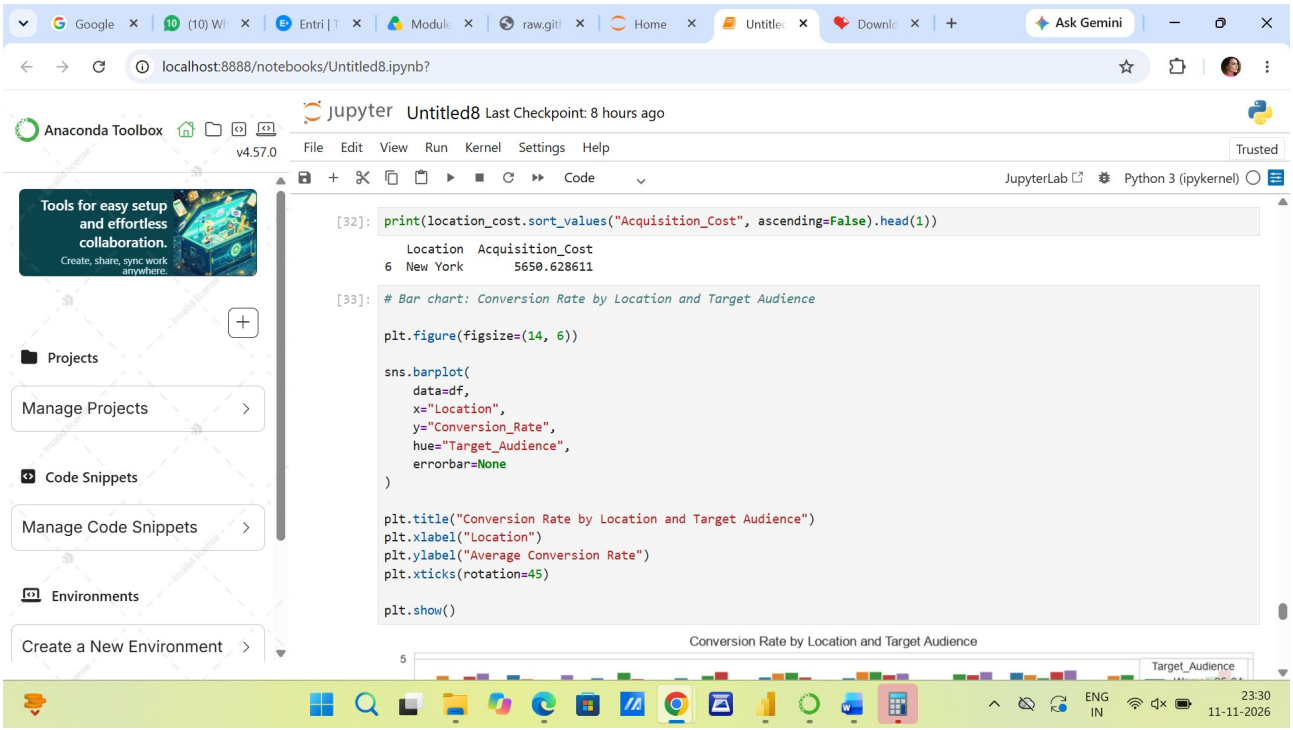


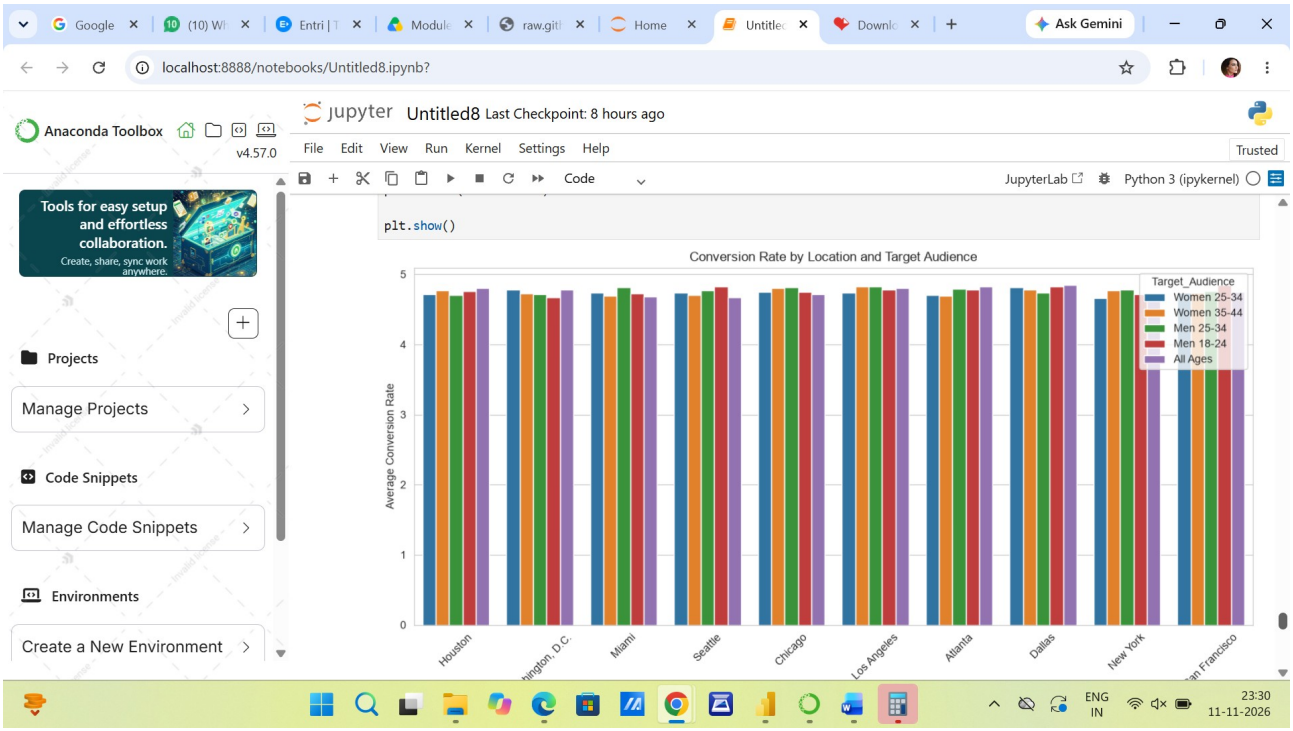
5.17 Bar Chart – Conversion Rate by Location and Target Audience

The grouped bar chart compares average conversion rate across locations and target audience groups.

```
plt.figure(figsize=(14, 6))
sns.barplot(
    data=df,
    x="Location",
    y="Conversion_Rate",
    hue="Target_Audience",
    errorbar=None
)
plt.title("Conversion Rate by Location and Target Audience")
```

```
plt.xlabel("Location")
plt.ylabel("Average Conversion Rate")
plt.xticks(rotation=45)
plt.show()
```

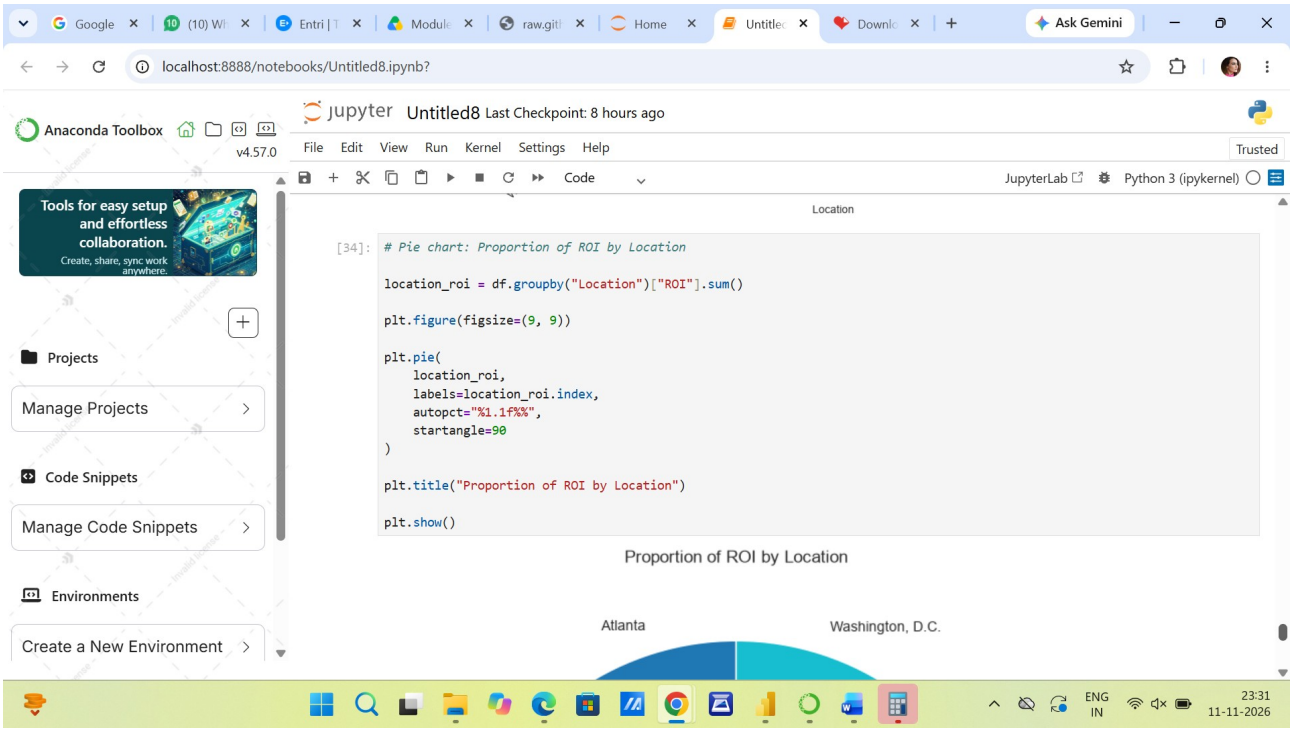


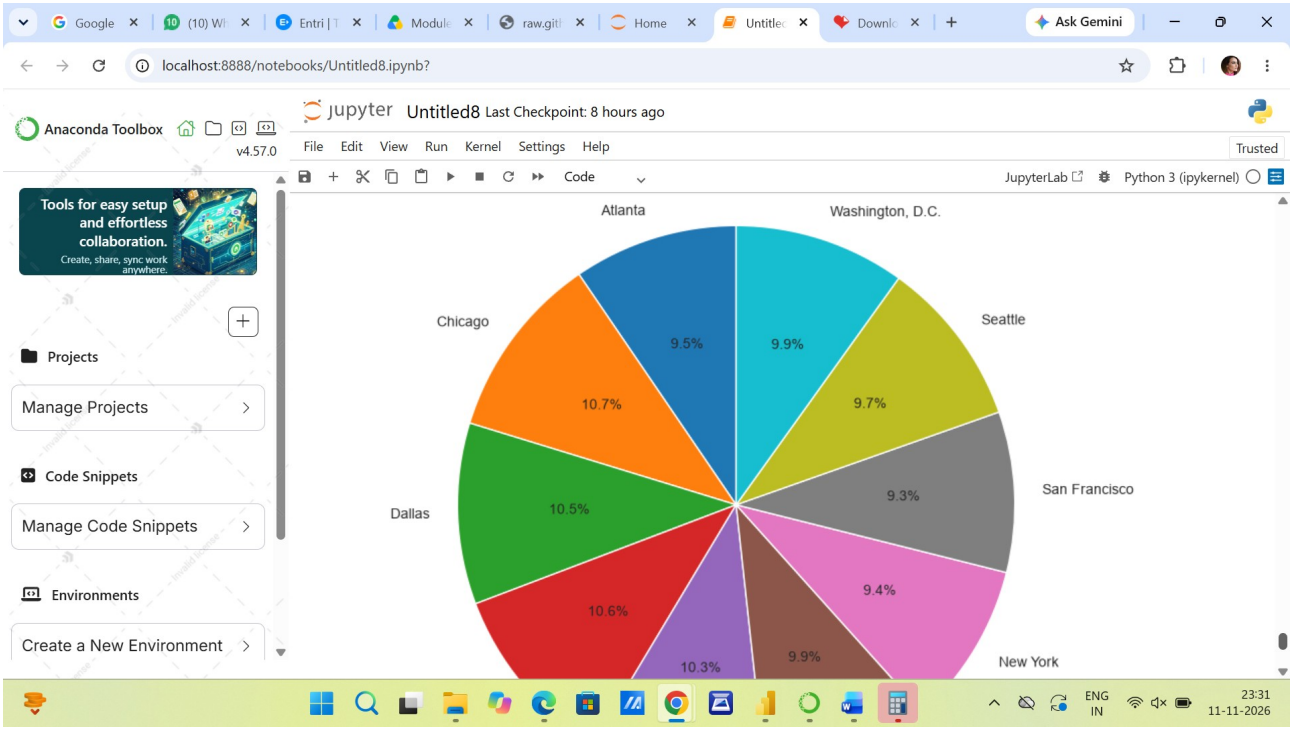


5.18 Pie Chart – Proportion of ROI by Location

Each pie slice represents a location's share of the total ROI.

```
location_roi = df.groupby("Location")["ROI"].sum()
plt.figure(figsize=(9, 9))
plt.pie(
    location_roi,
    labels=location_roi.index,
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Proportion of ROI by Location")
plt.show()
```





6. Conclusion

The project used Python to load and examine the marketing campaign dataset, summarize its structure and numerical characteristics, and create visualizations for campaign performance, customer segmentation, channel effectiveness, time-based trends and geographic analysis.

The descriptive analysis confirmed that the dataset contains 22,029 records and 16 columns. The EDA visualizations provide different ways to compare campaign outcomes and explore patterns in conversion rate, ROI, engagement, acquisition cost, clicks, impressions and campaign duration.

The analysis demonstrates how Pandas, Matplotlib and Seaborn can be used together to perform data exploration and communicate findings through visualizations.